

BECE102L – Digital Systems Design

Module 3 – Design of Combinational Logic Circuits

Dr. Rohit Mathur
Assistant Professor Sr. Grade 1
School of Electronics Engineering

Multiplexers

- Combinational Circuit to select a binary information from one of the many input lines to single output line. The term Multiplexer means many into one.
- Multiplexing is the process of transmitting a large number of information over a single line.
- A Multiplexer is also called a Data Selector.

➤ Multiplexer contains the followings:

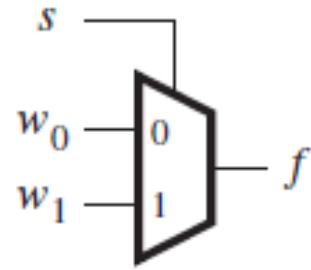
❖ data inputs

❖ selection inputs

❖ a *single output*

- Selection input determines the input that should be connected to the output.

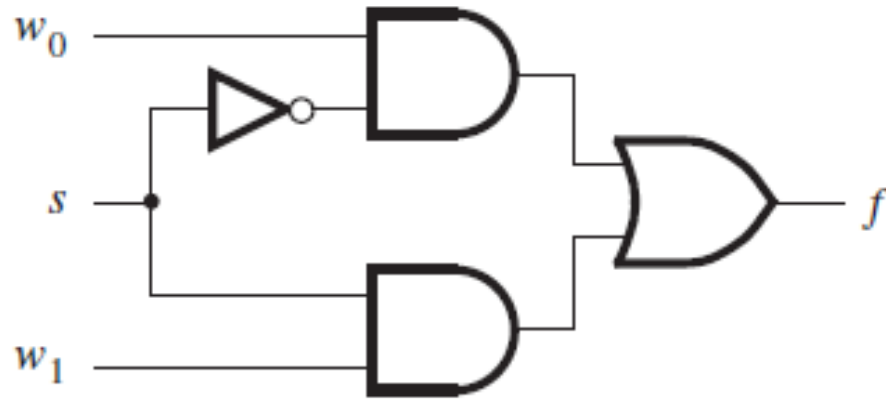
2:1 Multiplexer



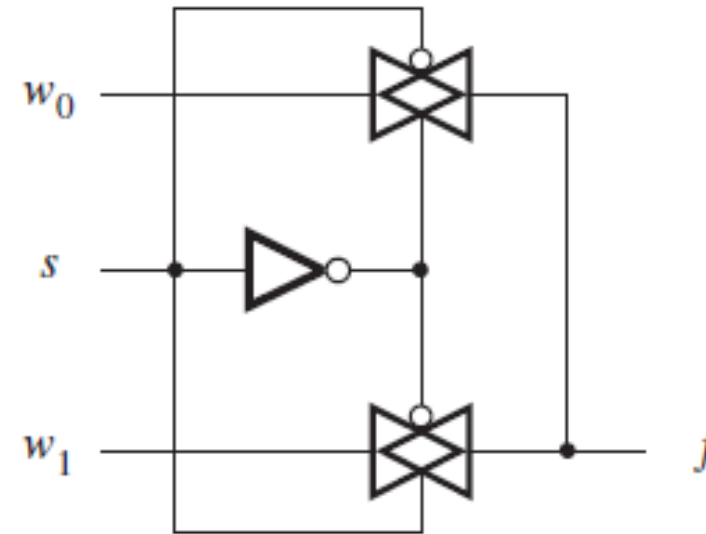
(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table



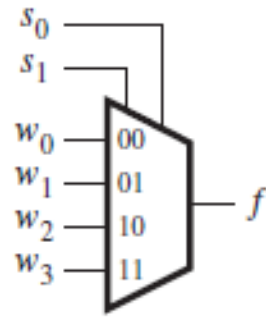
(c) Sum-of-products circuit



(d) Circuit with transmission gates

Figure 4.1 A 2-to-1 multiplexer.

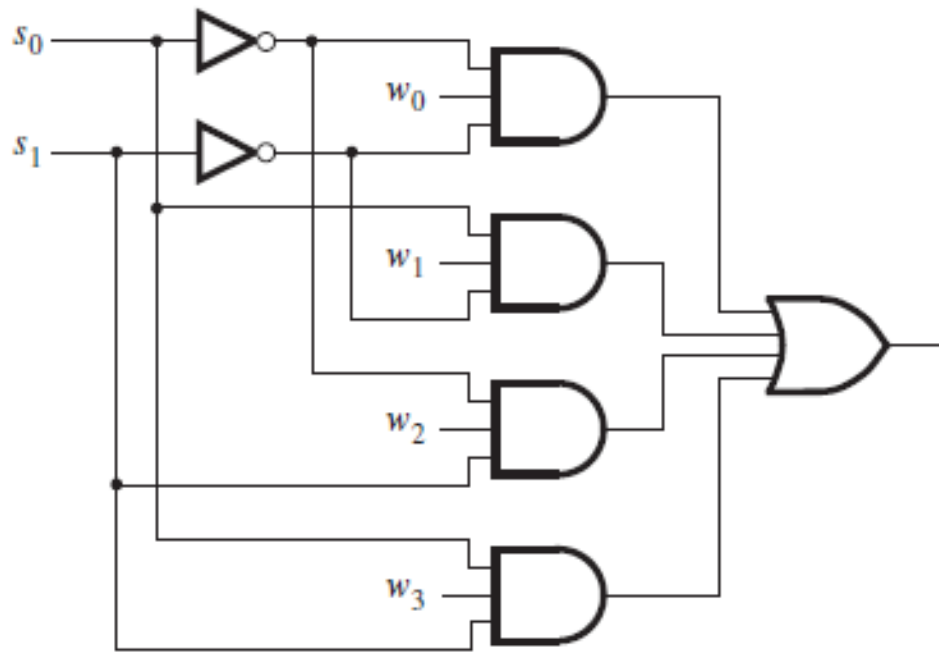
4:1 Multiplexer



(a) Graphical symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table



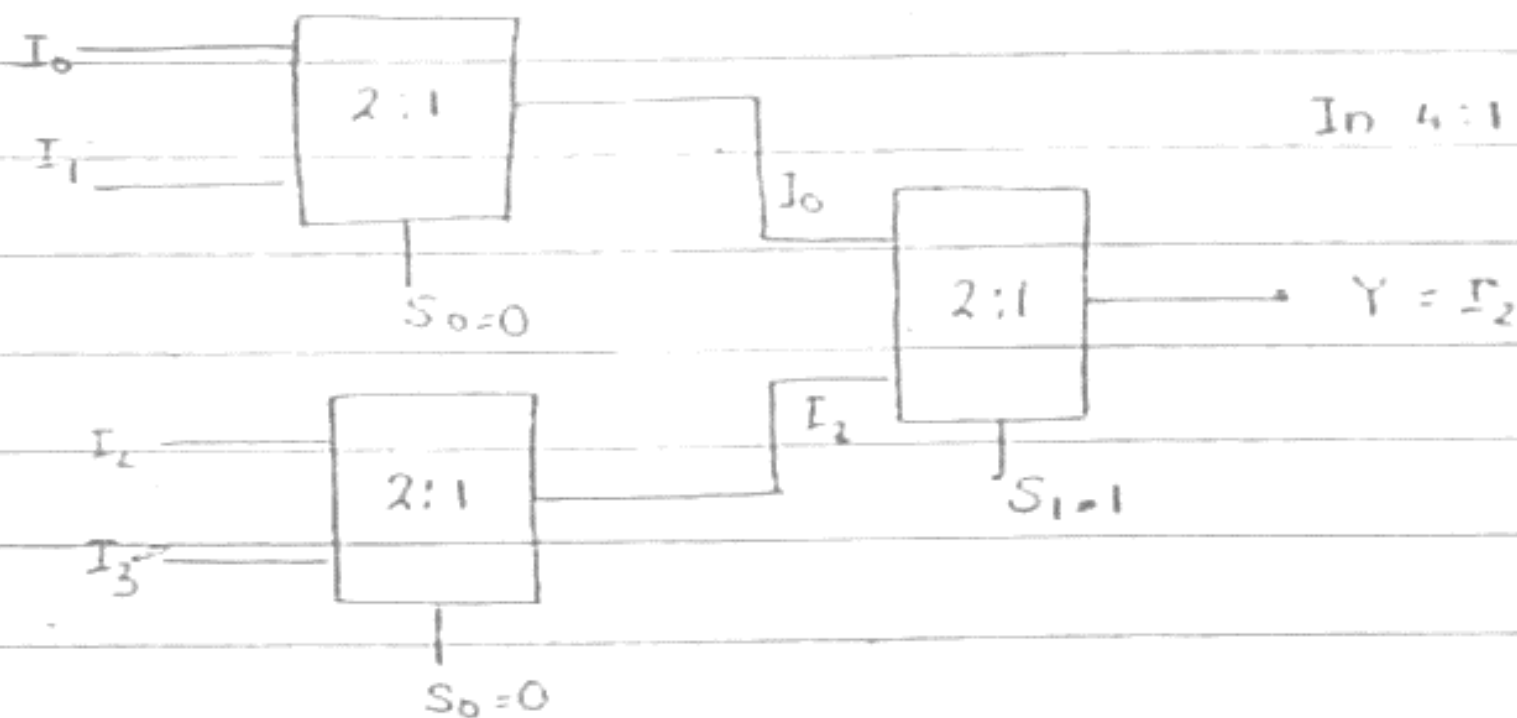
(c) Circuit

$$f = \bar{s}_1 \bar{s}_0 w_0 + \bar{s}_1 s_0 w_1 + s_1 \bar{s}_0 w_2 + s_1 s_0 w_3$$

Figure 4.2 A 4-to-1 multiplexer.

Implementation of higher order mux with lower order:-

Implement 4:1 MUX using 2:1 mux:-



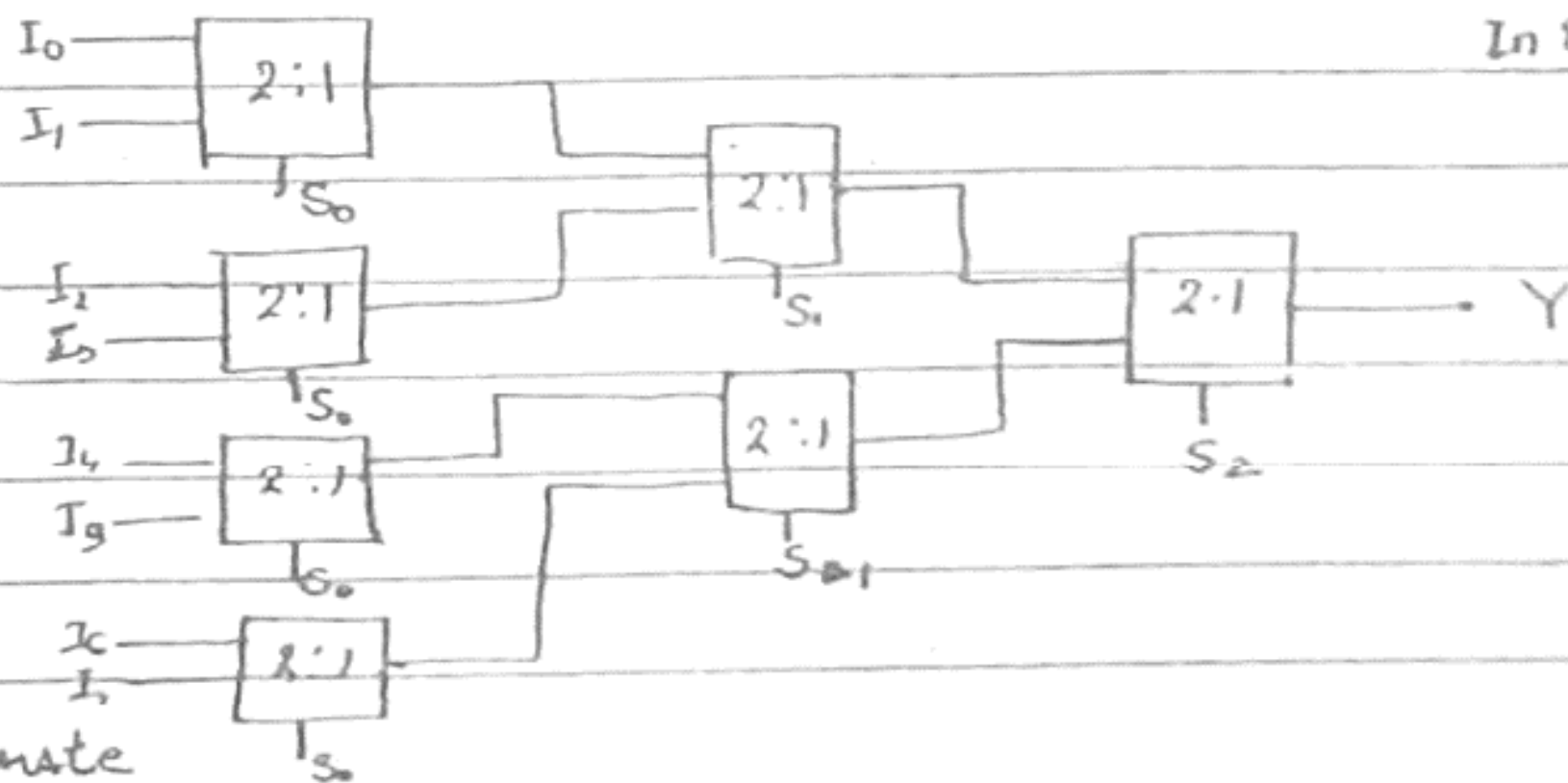
In 4:1, no. of 2:1 MUX = 3

$S_1 S_0$	Y
1 0	I_2

(B) Implement 8:1 MUX using 2:1 MUX:-

$$\text{no. of MUX} = \frac{8}{2} + \frac{4}{2} + \frac{2}{2} = 4 + 2 + 1 = 7$$

In 8:1, no. of 2:1 MUX = 7

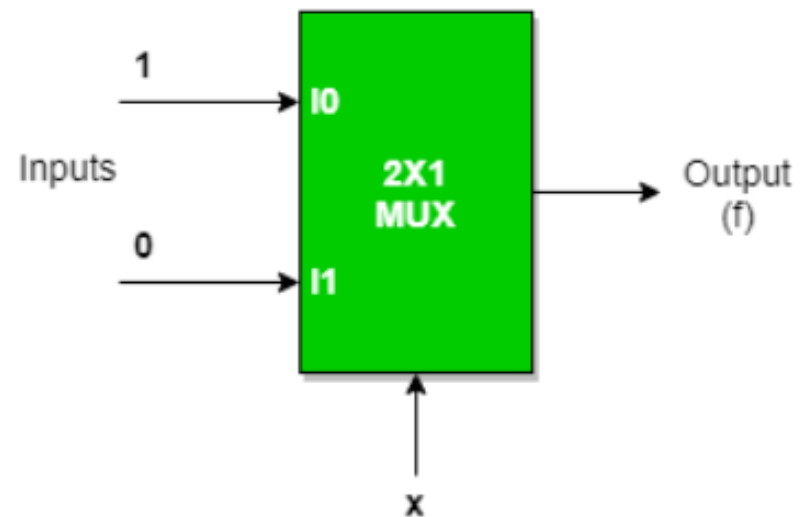


classmate

PAGE

--	--	--

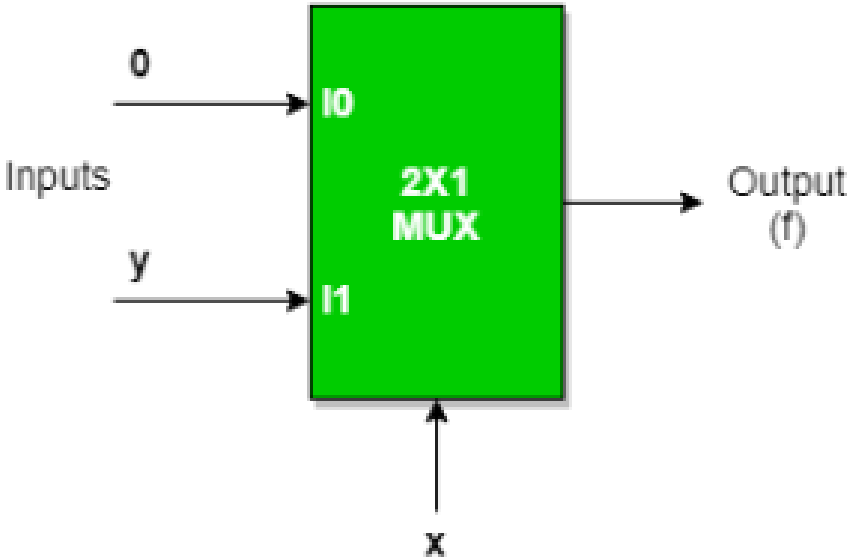
Implementation of NOT gate using 2 : 1 Mux



Truth Table

x	f
0	1
1	0

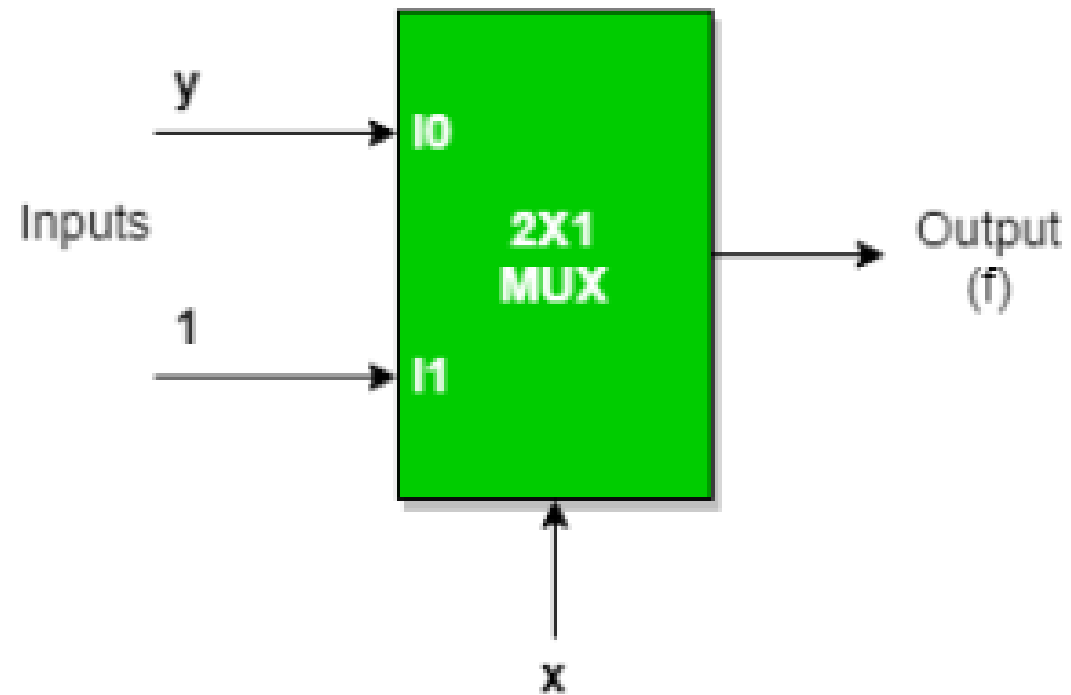
Implementation of AND gate using 2 : 1 Mux



Truth Table

x	y	f	$f \rightarrow y$
0	0	0	f = 0
0	1	0	
1	0	0	f = y
1	1	1	

Implementation of OR gate using 2 : 1 Mux

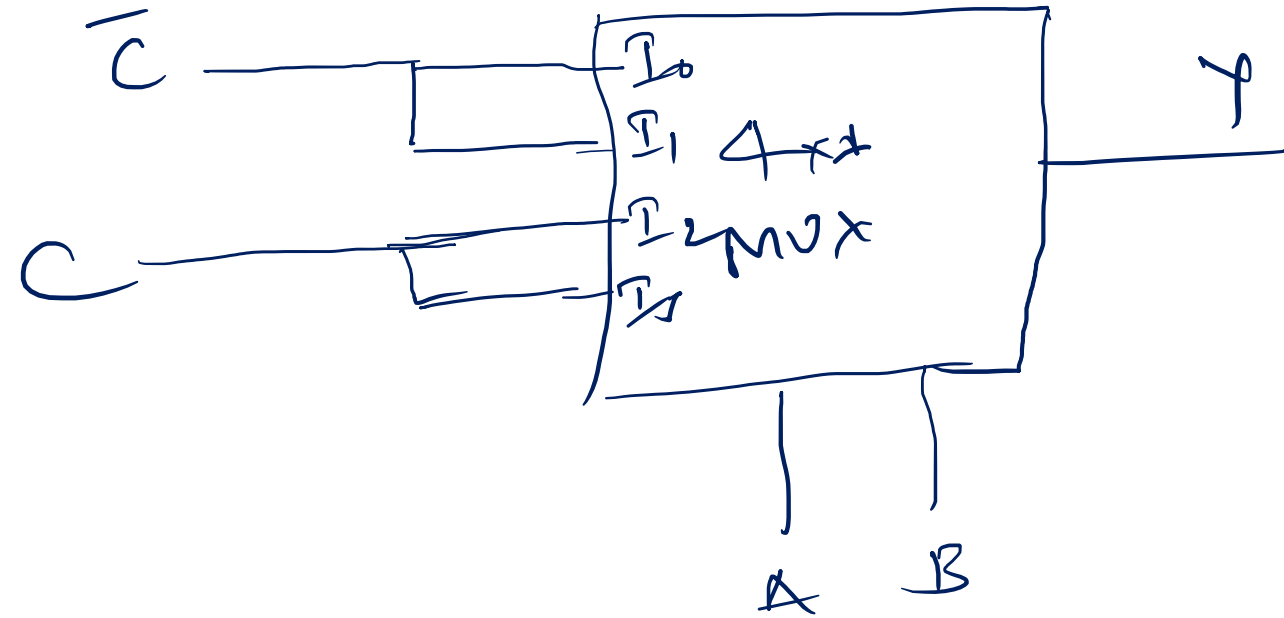


Truth Table

x	y	f	$f \rightarrow y$
0	0	0	$f = y$
0	1	1	
1	0	1	$f = 1$
1	1	1	

Model - III

(Determine the minimized logic expr)



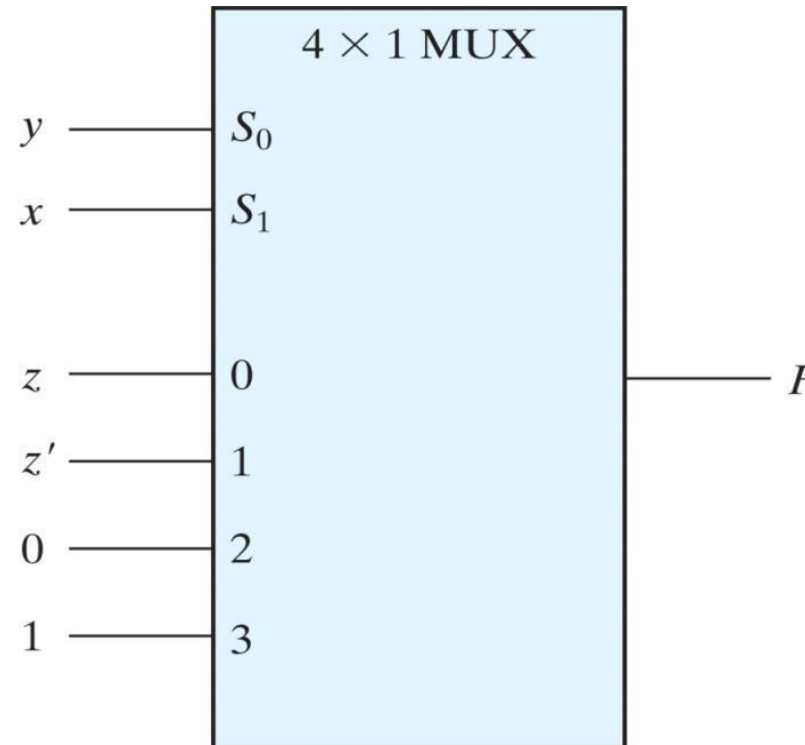
$$\begin{aligned} Y &= \bar{A} \bar{B} \bar{I}_0 + \bar{A} B \bar{I}_1 + A \bar{B} I_2 + A B I_3 \\ &= \bar{A} \bar{B} \bar{C} + \bar{A} B \bar{C} + A \bar{B} C + A B C \\ &= \bar{A} \bar{C} (B + \bar{B}) + A C (B + \bar{B}) \\ &= \bar{A} \bar{C} + A C = A \odot C \end{aligned}$$

Implementing Boolean Functions using Mux

- $F(x,y,z) = \Sigma(1,2,6,7)$
- x and y are connected to the selection inputs of the multiplexer in the same order.
- x is connected to S_1 and y is connected to S_0
- $z, z', 0$ and 1 are connected to the data inputs of the multiplexer

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

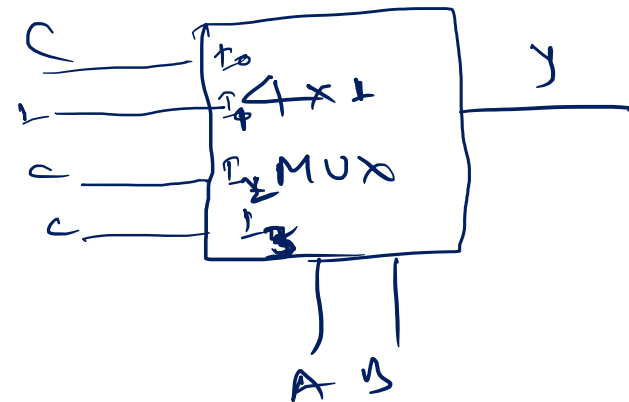
(a) Truth table



(b) Multiplexer implementation

$$F(A, B, C) = m(1, 2, 3, 5, 7)$$

- i) $\overline{A}B$ as select line
- ii) AC as select line.
- iii) BC as a select line.

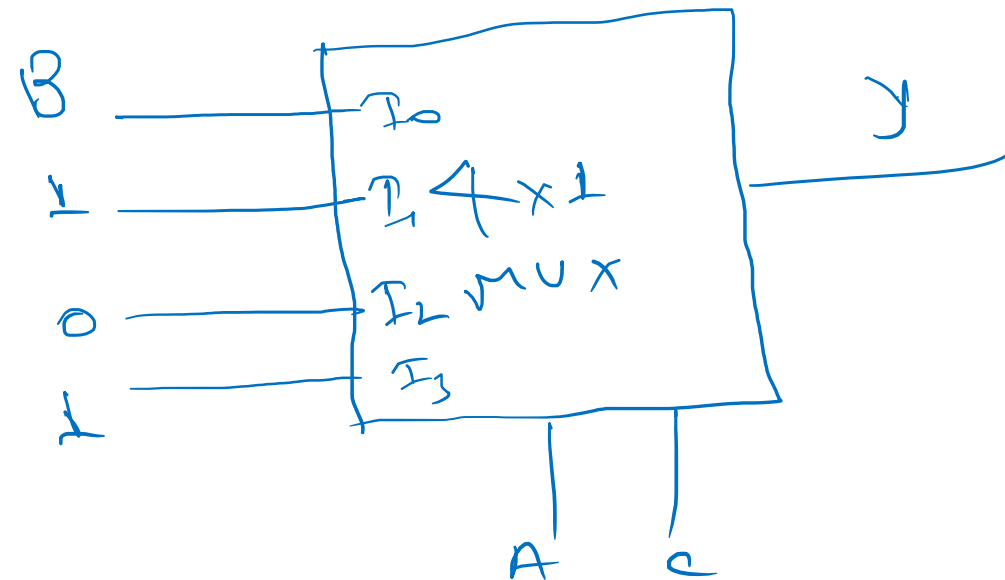


ii) AC as a select line

	I_0	I_1	I_2	I_3
$\overline{C}$	0	2	4	6
C	1	3	5	7
	C	L	C	C

ii) AC as a select line

	I_0	I_1	I_2	I_3
$\bar{B}$	0	1	4	5
B	2	3	6	7
	B	L	0	L

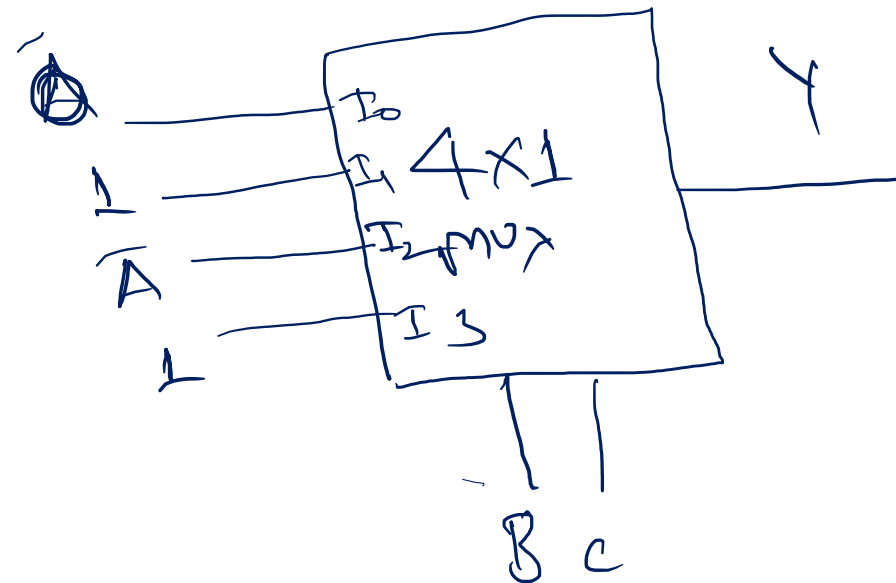


	A	B	C	$\bar{B}$
0	0	0	0	$\bar{B}$
1	0	0	1	$\bar{B}$
2	0	1	0	B
3	0	1	1	B
4	1	0	0	$\bar{B}$
5	1	0	1	B
6	1	1	0	B
7	1	1	1	B

iii) BC as a select line.

Implementation table

	I_0	I_1	I_2	I_3
$\bar{A}$	0	1	2	3
A	4	5	6	7
	0	1	$\bar{A}$	1



	A	B	C	
$\bar{A}$	0	0	0	0
$\bar{A}$	0	0	1	1
$\bar{A}$	0	1	0	2
$\bar{A}$	0	1	1	3
A	1	0	0	4
A	1	0	1	5
A	1	1	0	6
A	1	1	1	7

Applicat:- Implementation of Boolean function using Multiplexers.

Implement :- $F(A, B, C, D) = \sum m(1, 4, 5, 7, 9, 12, 13)$ using 4×1 Mux

Applicat:- Implementation of Boolean function using Multiplexers.

Implement :- $F(A, B, C, D) = \sum m(1, 4, 5, 7, 9, 12, 13)$ using 4×1 Mux

AB \ CD				
	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	CD 11	$C\bar{D}$ 10
$\bar{A}\bar{B}$ 00				
$\bar{A}B$ 01				
AB 11				
$A\bar{B}$ 10				

Two part given

→ min terms

→ 4×1 MUX (Imp)

Step-1 :- Write K-Map based on variable involved in the function

Step-2 :- Use 4×1 MUX

Step-3 :- Define selection line using variables.

Step-4 :- write mux (4×1) table

Step-5 :- Fill K-Map

Step-6 :- K-Map & truth table define I/p & o/p function of 4×1 MUX.

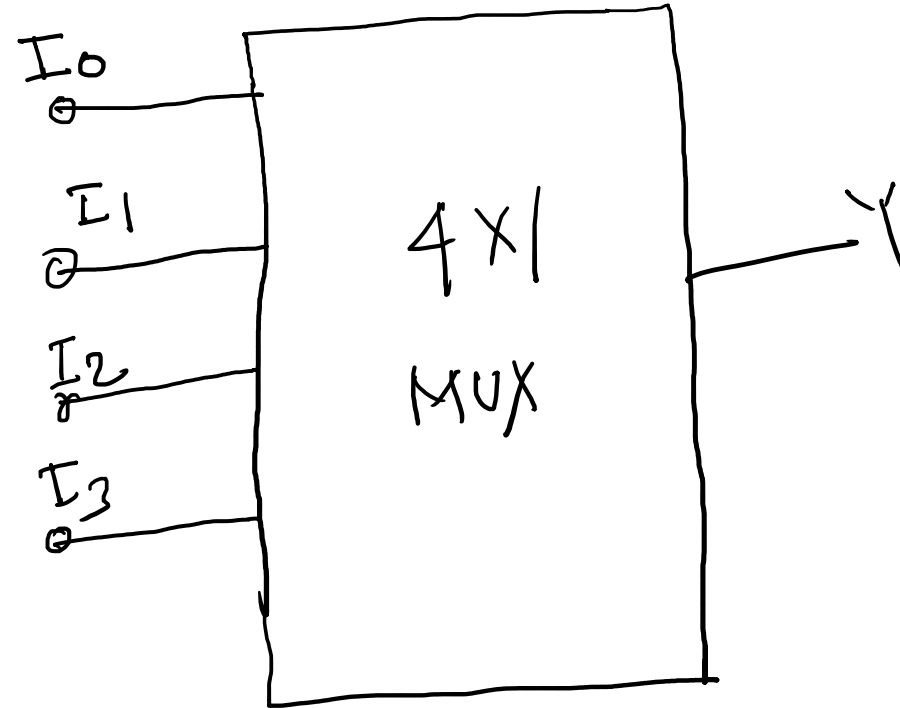
Step-1 $\rightarrow$ K-Map.

AB \ CD				
	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	CD 11	$C\bar{D}$ 10
$\bar{A}\bar{B}$ 00				
$\bar{A}B$ 01				
AB 11				
$A\bar{B}$ 10				

Step-1 $\rightarrow$ K-Map.

AB \ CD	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	$C\bar{D}$ 11	CD 10
$\bar{A}\bar{B}$ 00				
$\bar{A}B$ 01				
AB 11				
$A\bar{B}$ 10				

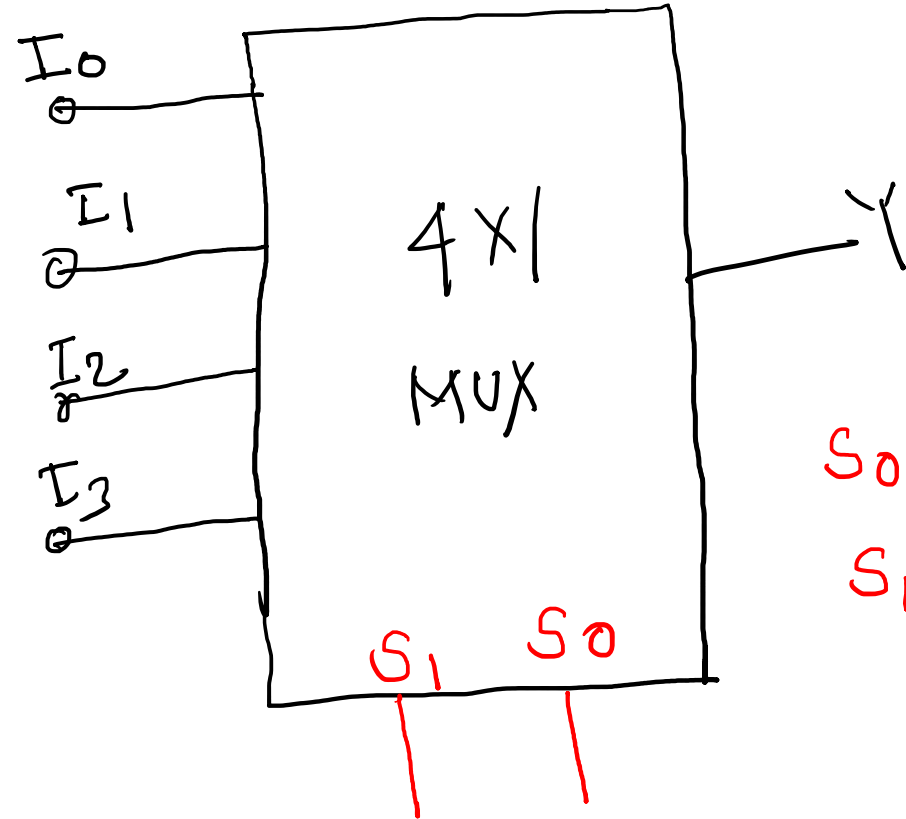
Step-2 $\rightarrow$ 4x1 MUX



Step-1 $\rightarrow$ K-Map.

AB \ CD	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	$C\bar{D}$ 11	CD 10
$\bar{A}\bar{B}$ 00				
$\bar{A}B$ 01				
AB 11				
$A\bar{B}$ 10				

Step-2 $\rightarrow$ 4x1 MUX

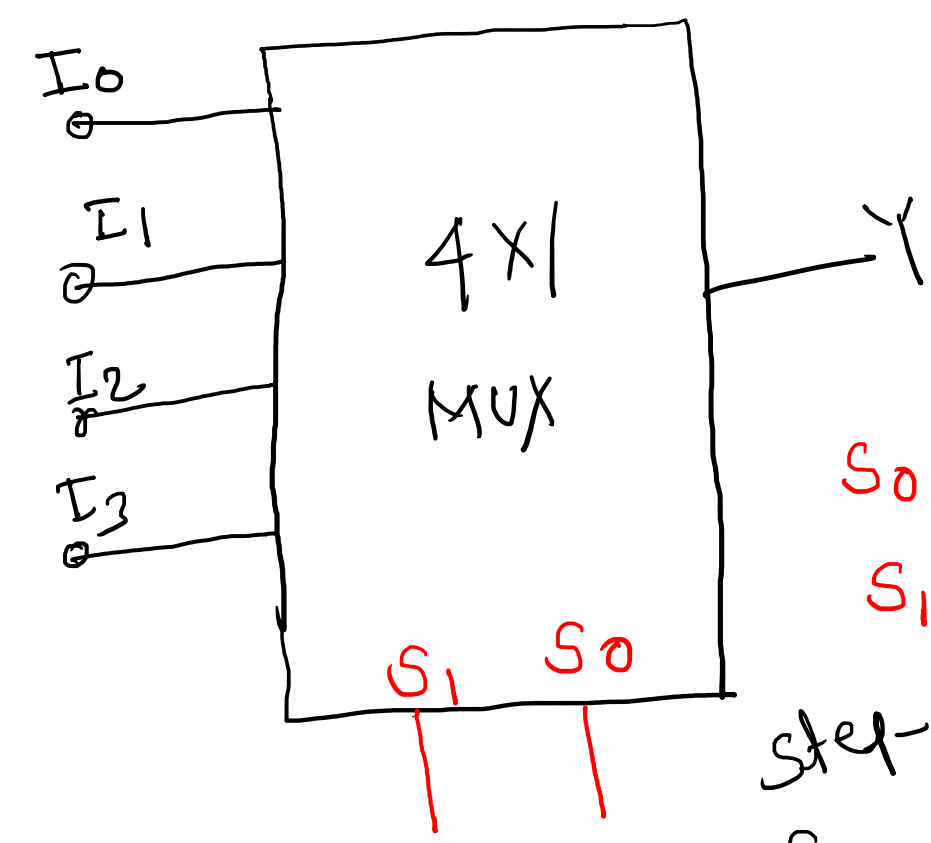


$S_0 = B$
 $S_1 = A$ } $SL \rightarrow$ Step-3

Step-1 $\rightarrow$ K-Map.

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$ 00				
$\bar{A}B$ 01				
AB 11				
$A\bar{B}$ 10				

Step-2 $\rightarrow$ 4x1 MUX



$S_0 = B$
 $S_1 = A$ } $S_L \rightarrow$ Step-3

Step-4.

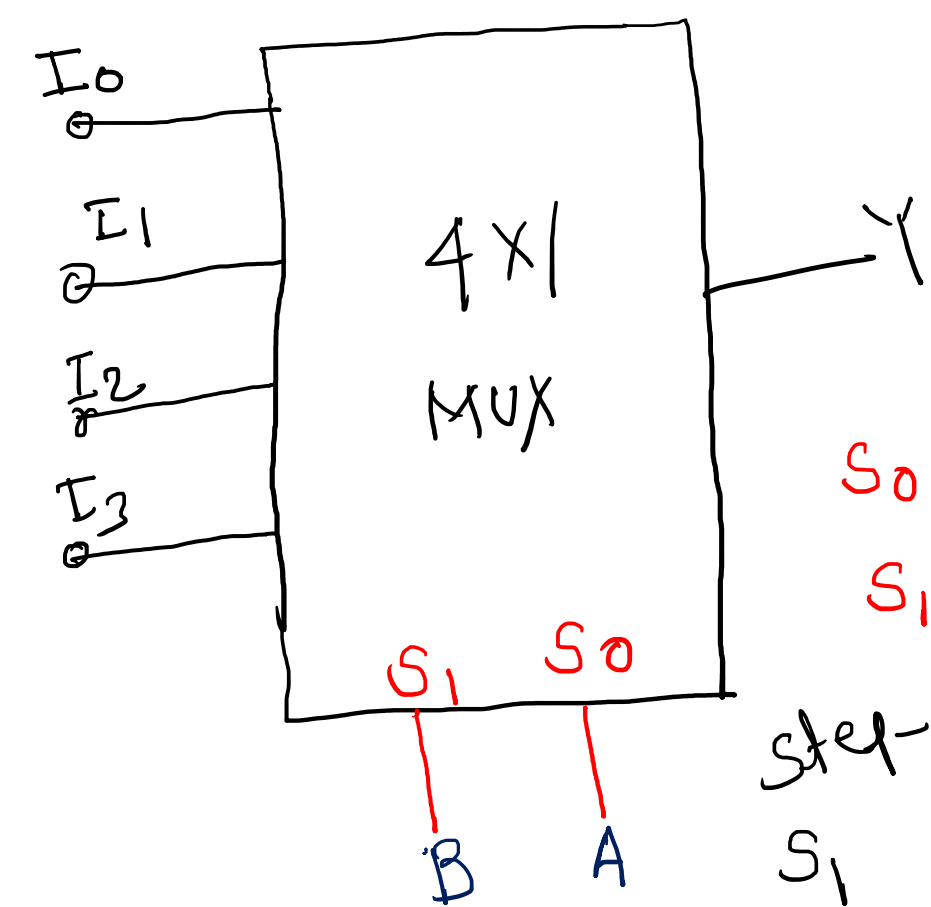
S_1	S_0	Y
0	0	
0	1	
1	0	
1	1	

Step-1 $\rightarrow$ K-Map.

$S_0 S_1$

AB	CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$ 00	0	1	0	0	
$\bar{A}B$ 01	1	1	1		
AB 11	1	1			
$A\bar{B}$ 10		1			

Step-2 $\rightarrow$ 4x1 MUX



$S_0 = B$
 $S_1 = A$
 $S_L \rightarrow \text{Step-3}$

Step-4.

S_1	S_0	Y
0	0	0
0	1	1
1	0	1
1	1	1

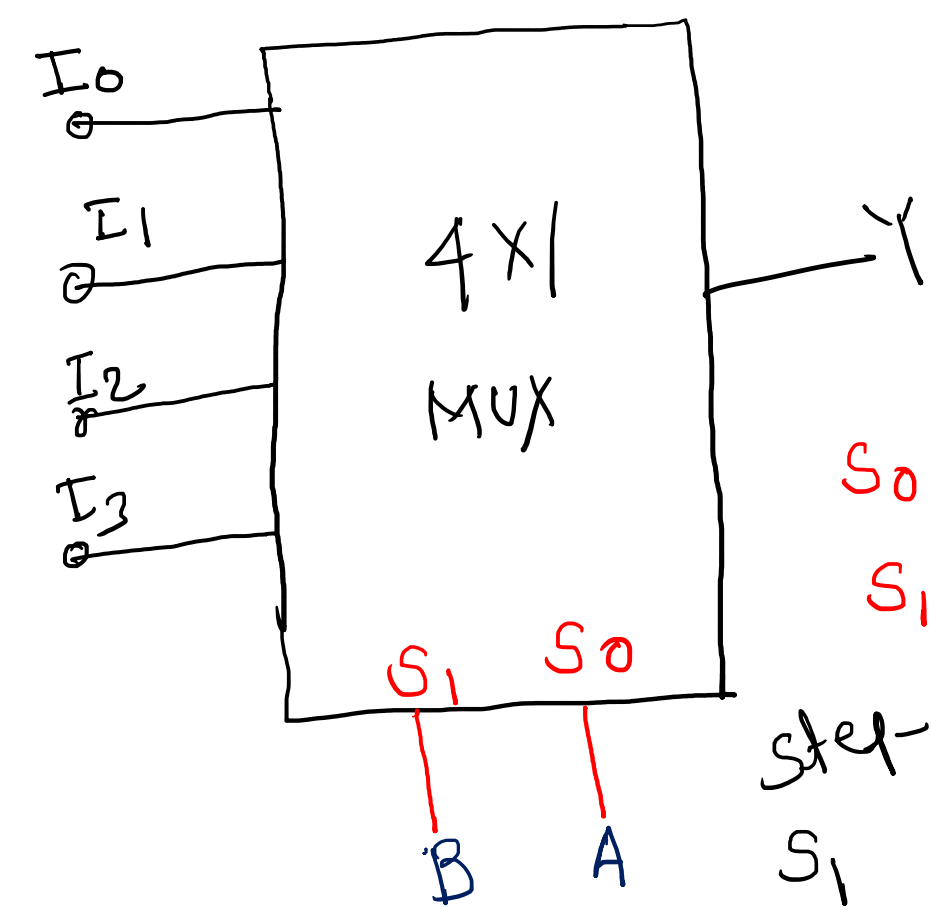
Step-1 $\rightarrow$ K-Map.

$S_0 S_1$

AB CD

	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	$C\bar{D}$ 11	CD 10	
$\bar{A}\bar{B}$ 00	ϕ	ϕ	ϕ	ϕ	$\bar{C}D=10$
$\bar{A}B$ 01	1	1	1		
AB 11	1	1			
$A\bar{B}$ 10		1			

Step-2 $\rightarrow$ 4x1 MUX

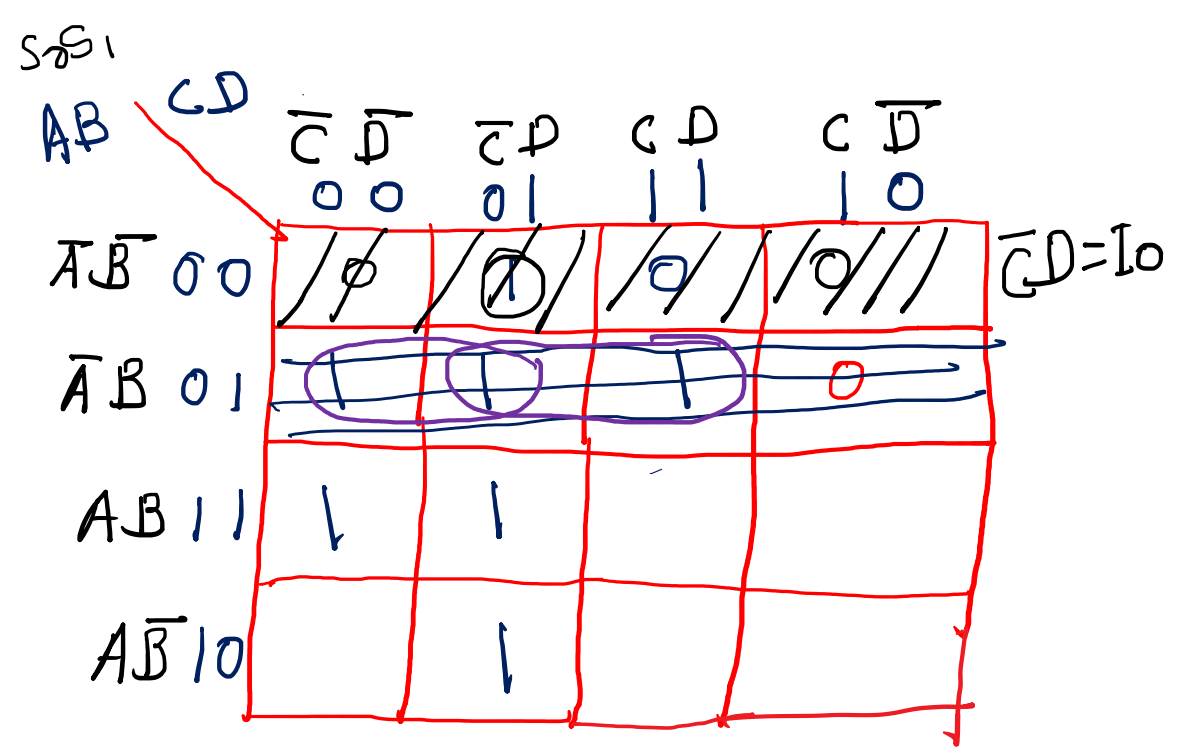


$S_0 = B$
 $S_1 = A$ } $S_L \rightarrow$ Step-3

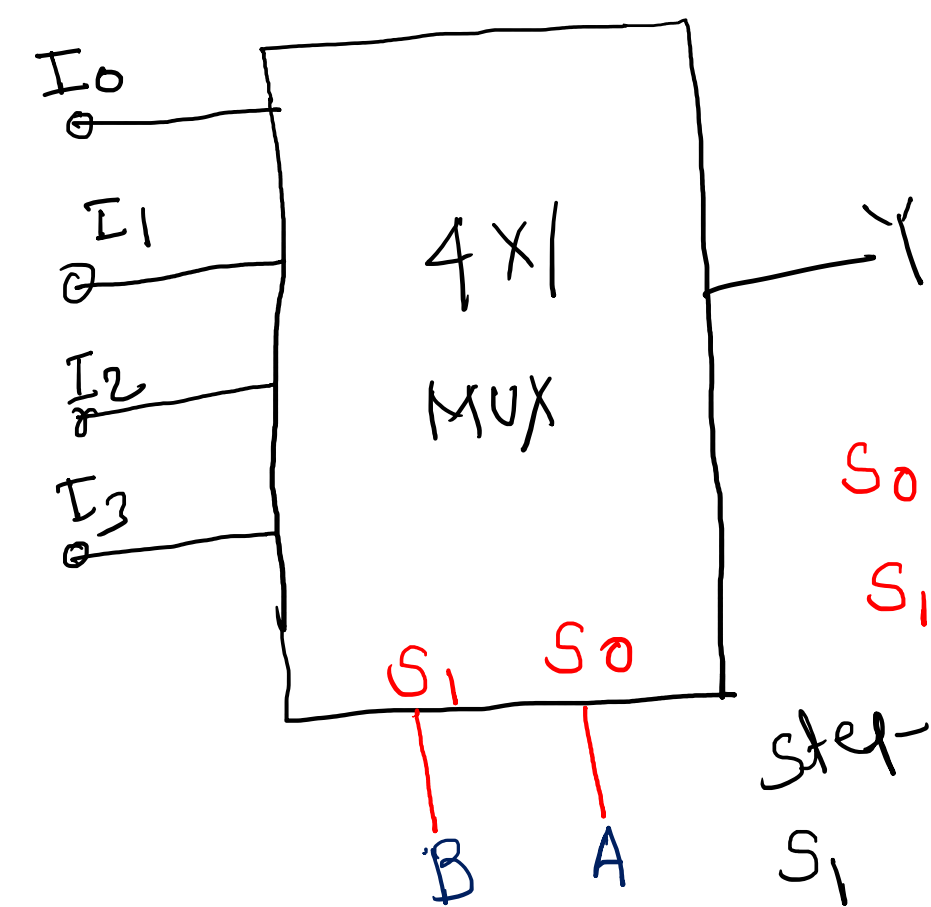
Step-4.

S_1	S_0	Y
0	0	
0	1	
1	0	
1	1	

Step-1 $\rightarrow$ K-Map.

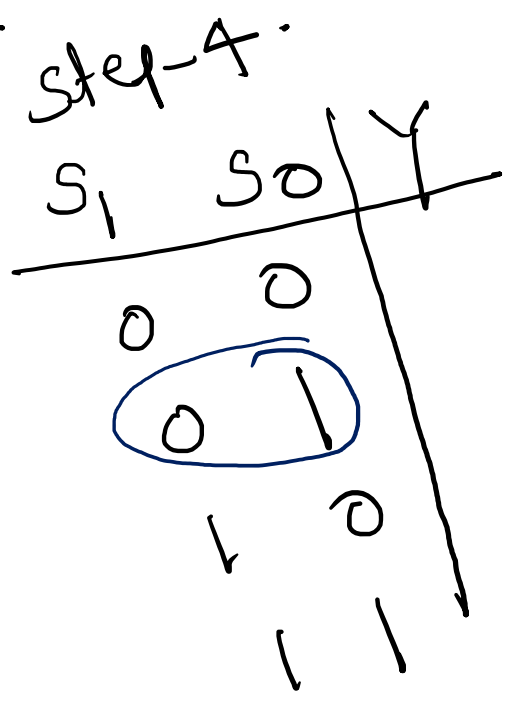


Step-2 $\rightarrow$ 4x1 MUX

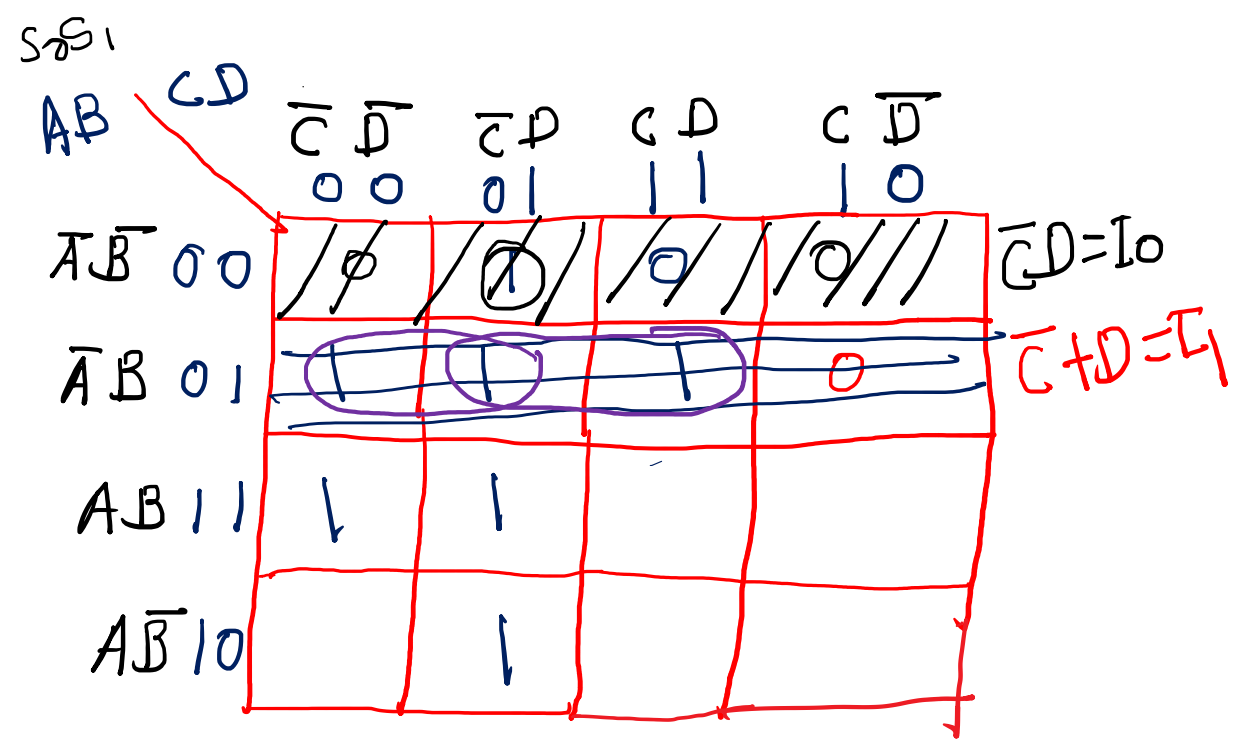


$S_0 = B$
 $S_1 = A$

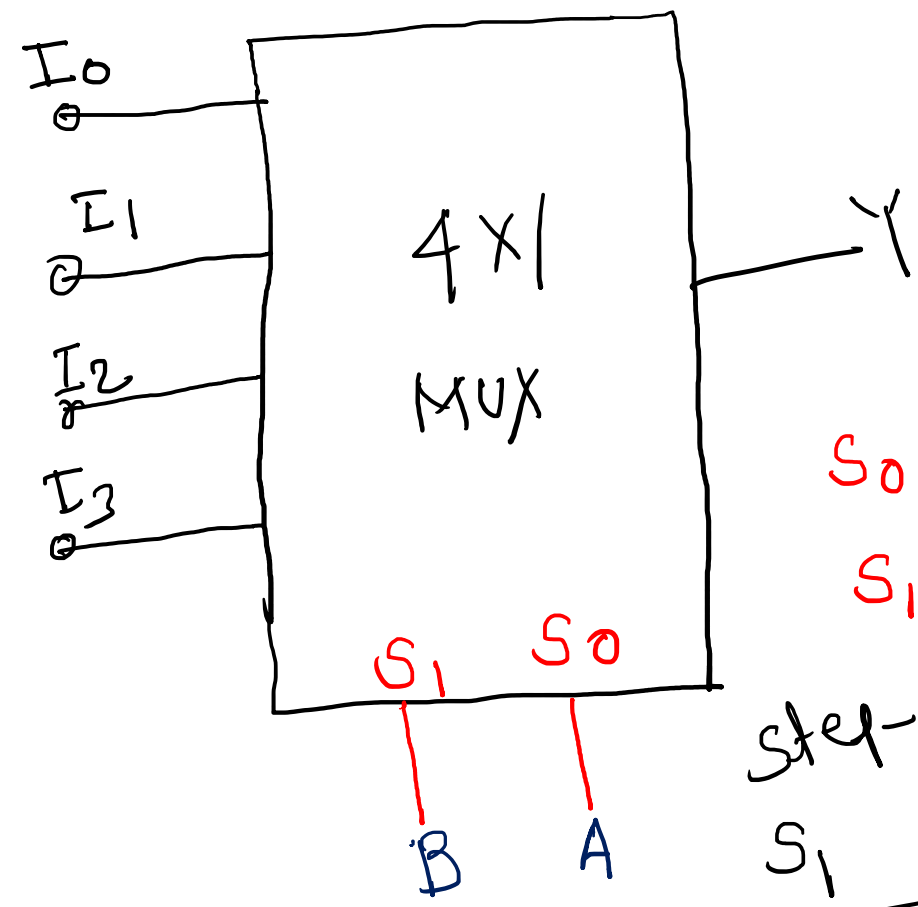
$S_L \rightarrow$ Step-3



Step-1 $\rightarrow$ K-Map.

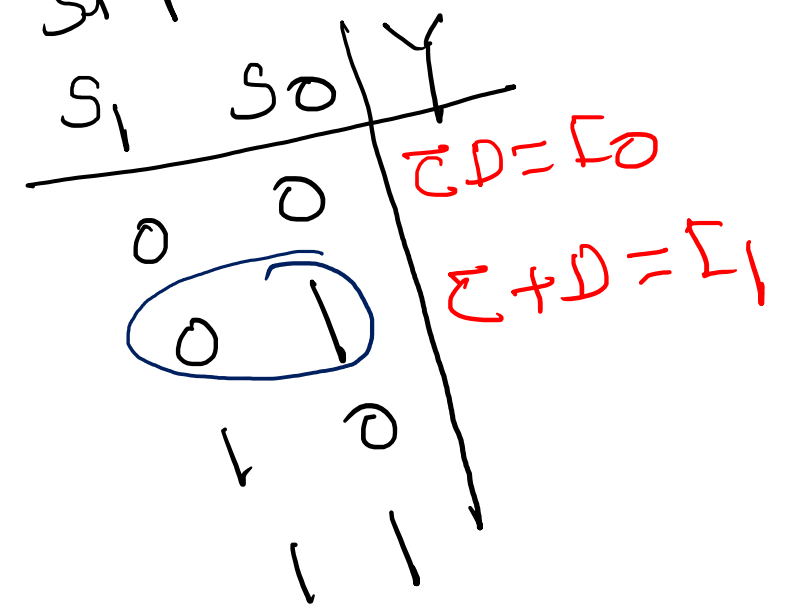


Step-2 $\rightarrow$ 4x1 MUX



$S_0 = B$
 $S_1 = A$

Step-4.



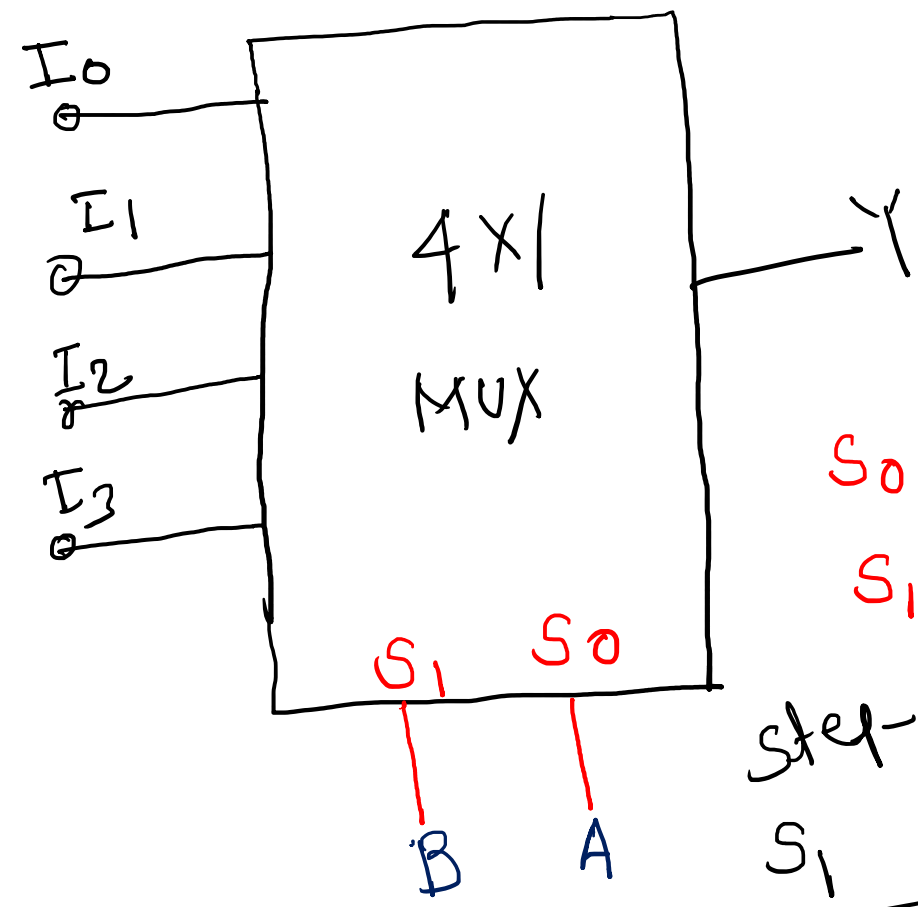
Step-1 $\rightarrow$ K-Map.

$S_1 S_0$

AB	CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	00	/	/	/	/
$\bar{A}B$	01	1	1	1	0
AB	11	1	1	0	0
$A\bar{B}$	10	0	1	0	0

$\bar{C}D = I_0$
 $\bar{C} + D = I_1$

Step-2 $\rightarrow$ 4x1 MUX



$S_0 = B$
 $S_1 = A$

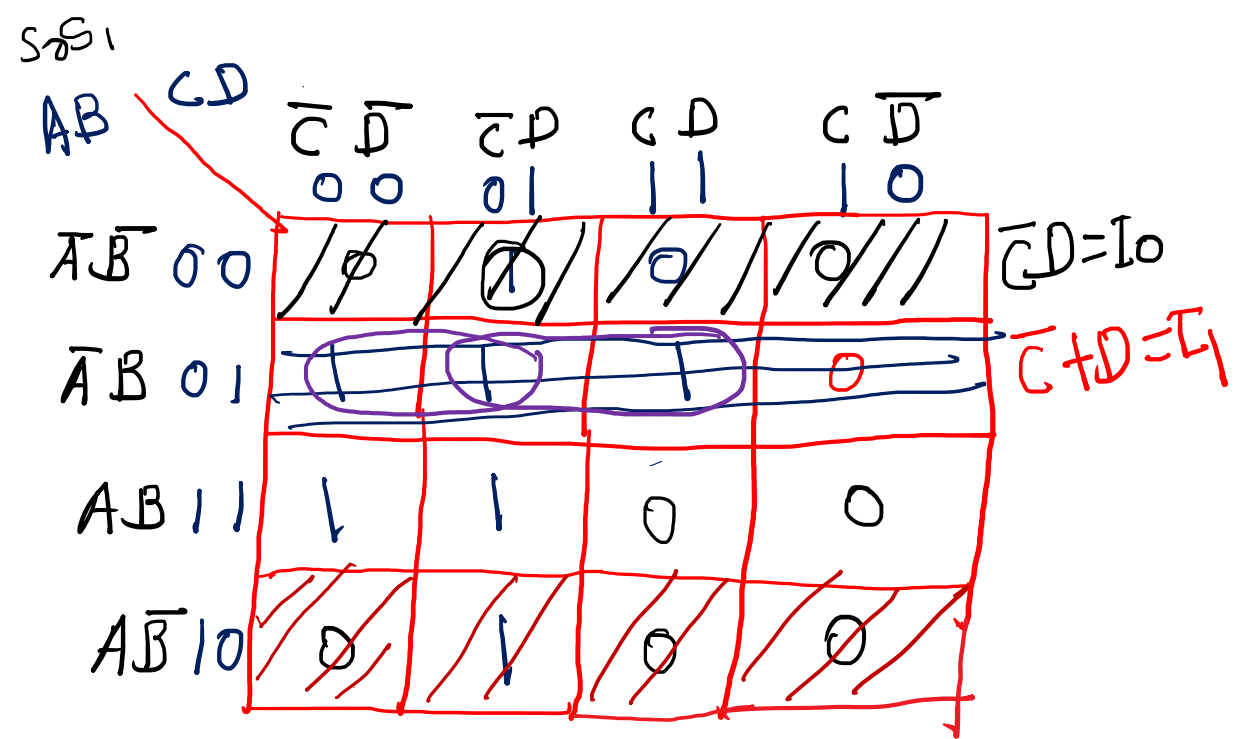
$S_L \rightarrow$ Step-3

Step-4.

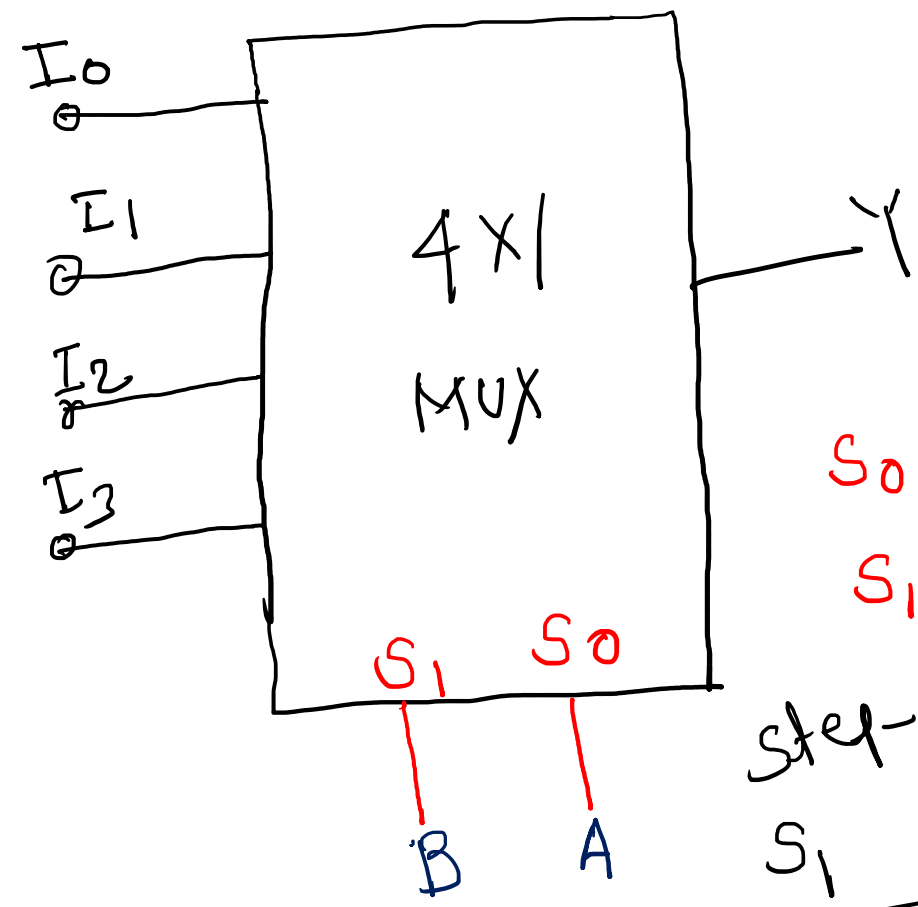
S_1	S_0	Y
0	0	0
0	1	1
1	0	1

$\bar{C}D = I_0$
 $\bar{C} + D = I_1$

Step-1 $\rightarrow$ K-Map.

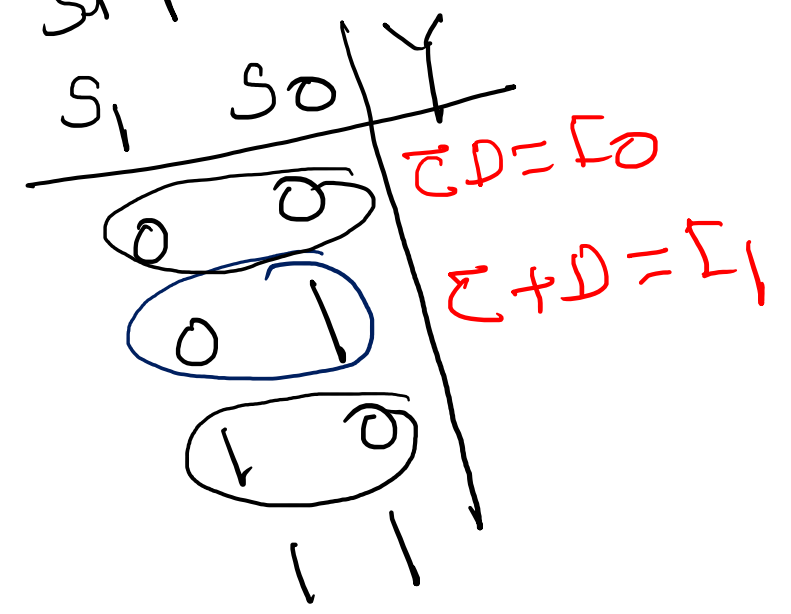


Step-2 $\rightarrow$ 4x1 MUX



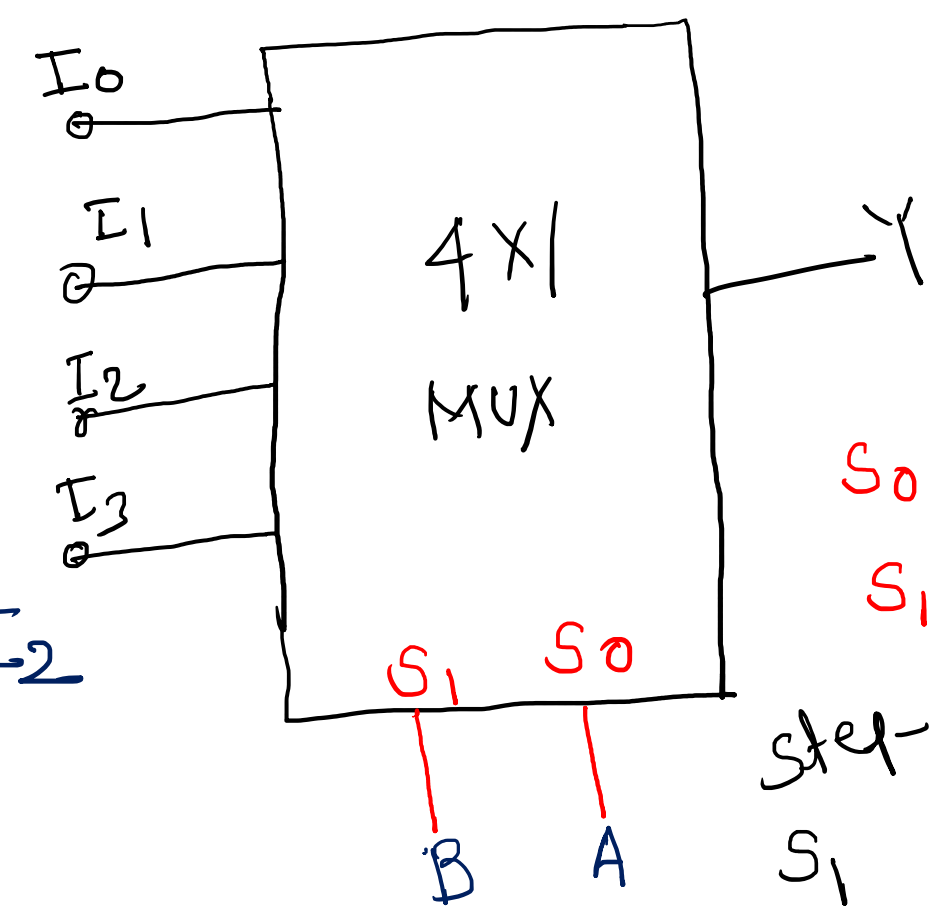
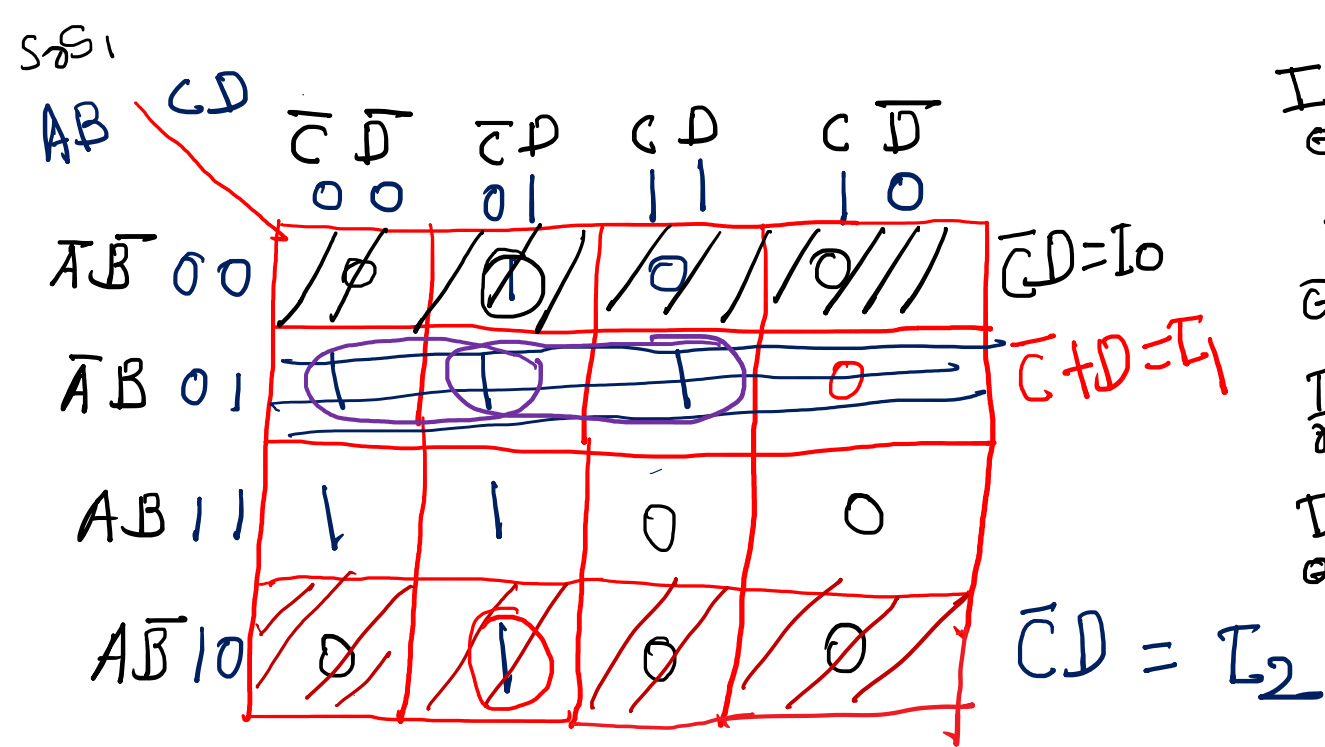
$S_0 = B$
 $S_1 = A$

Step-4.

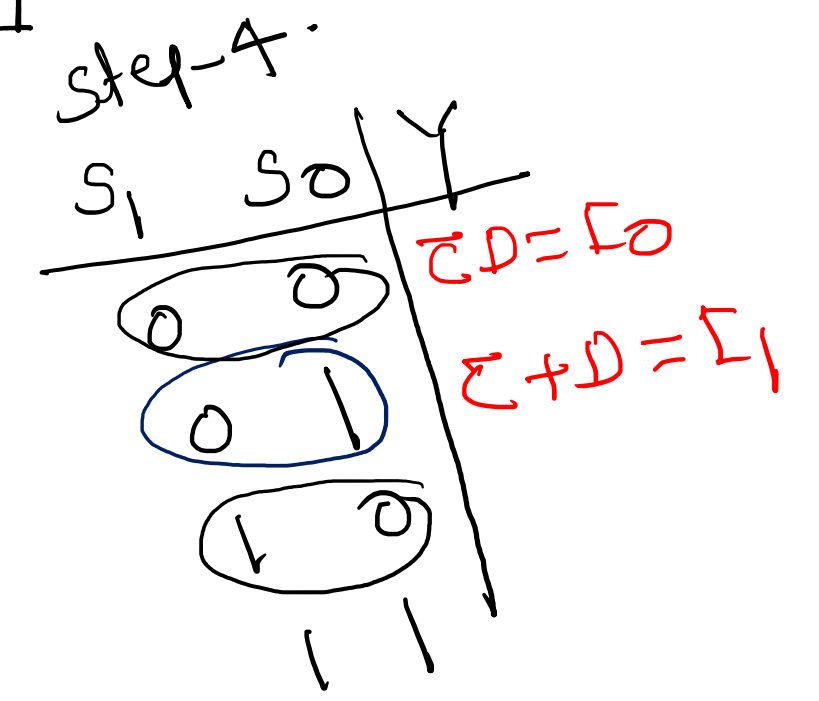


Step-1 $\rightarrow$ K-Map.

Step-2 $\rightarrow$ 4x1 MUX

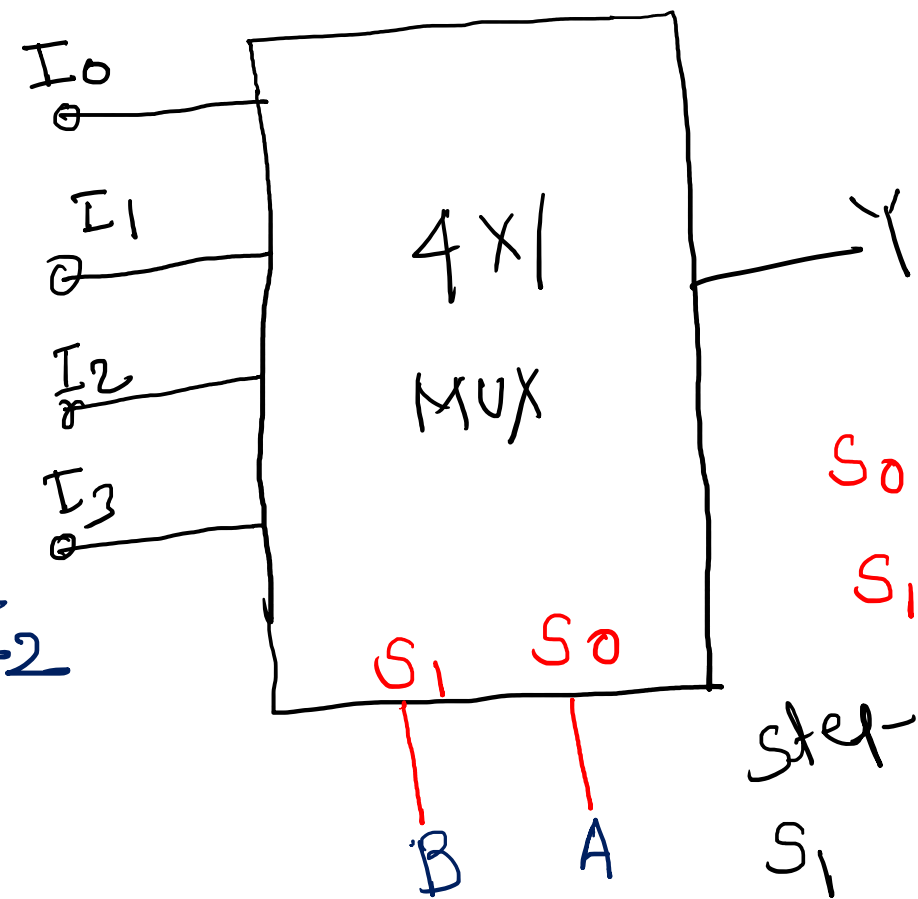
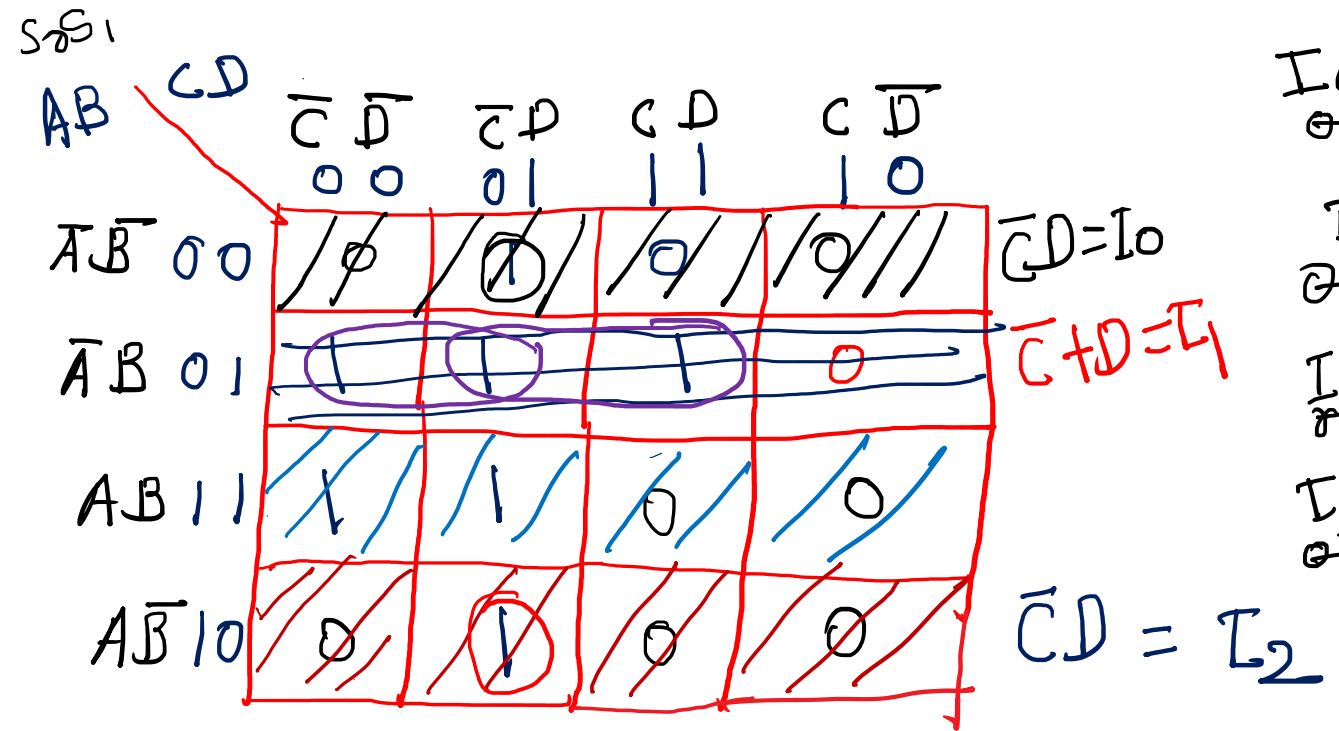


$S_0 = B$
 $S_1 = A$
 $S_L \rightarrow \text{Step-3}$



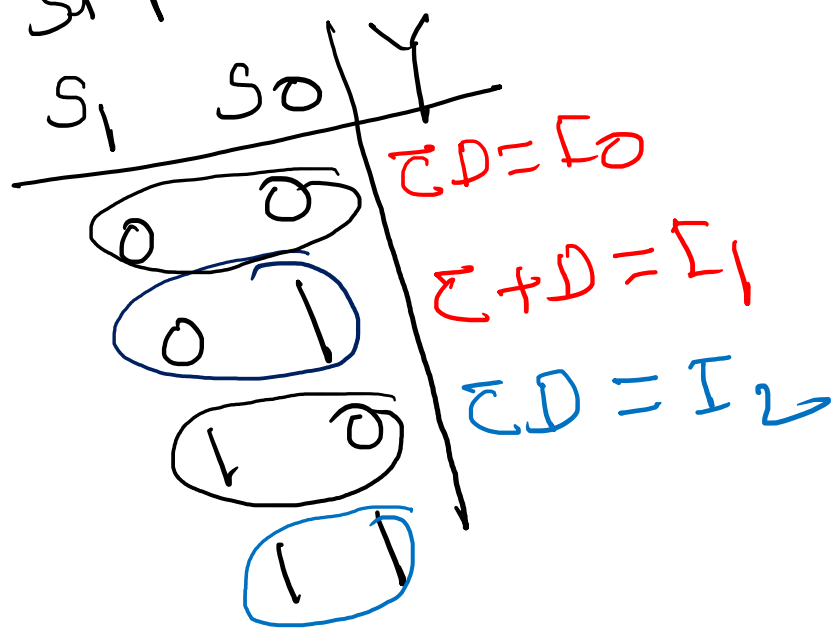
Step-1 $\rightarrow$ K-Map.

Step-2 $\rightarrow$ 4x1 MUX



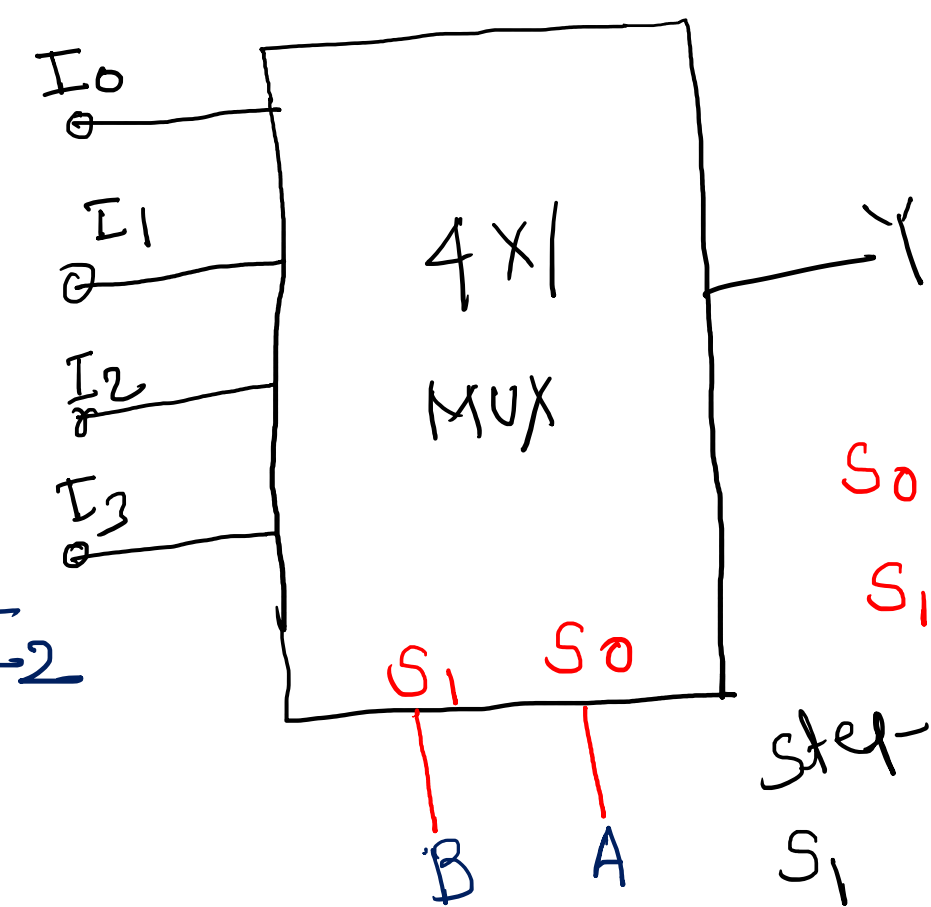
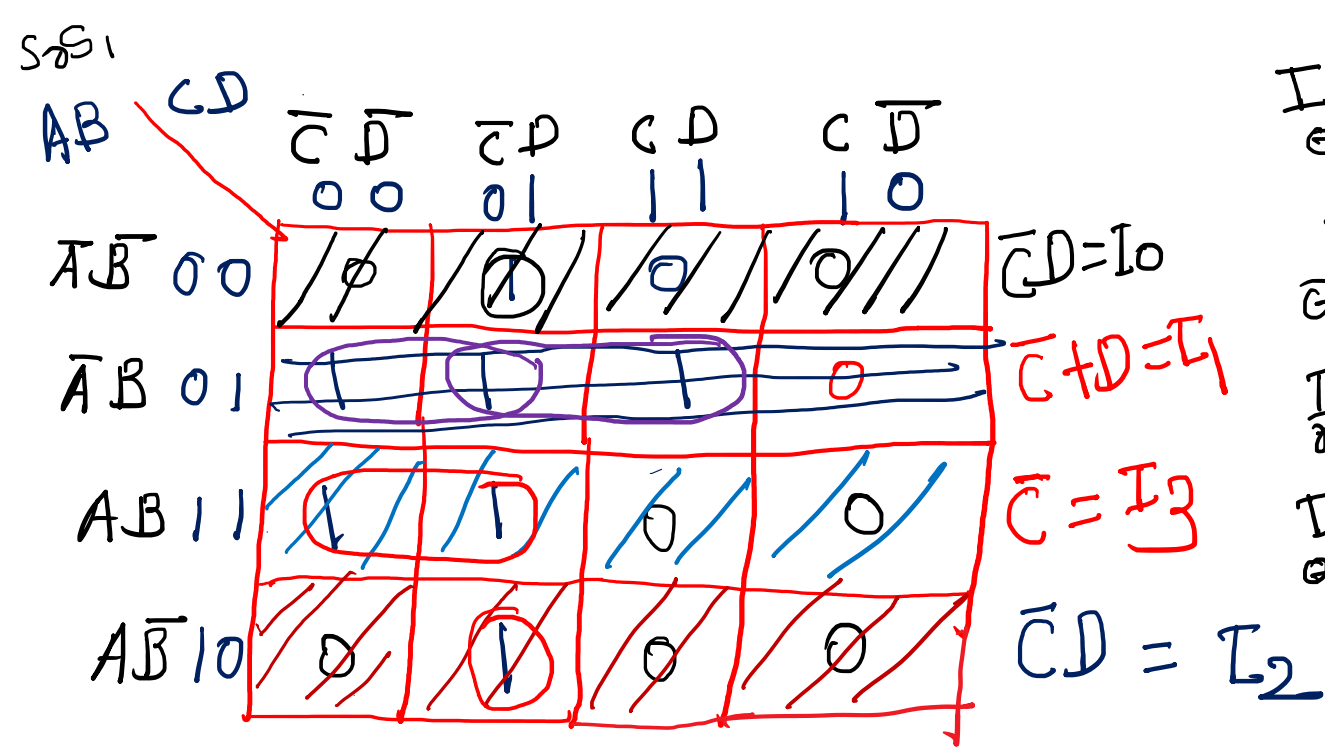
$S_0 = B$
 $S_1 = A$

Step-4.

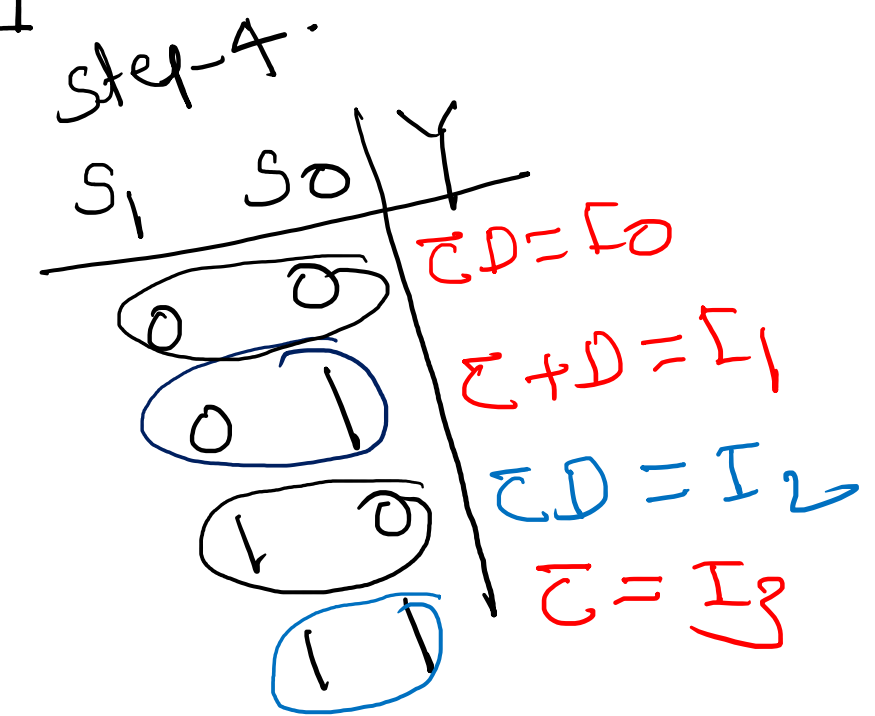


Step-1 $\rightarrow$ K-Map.

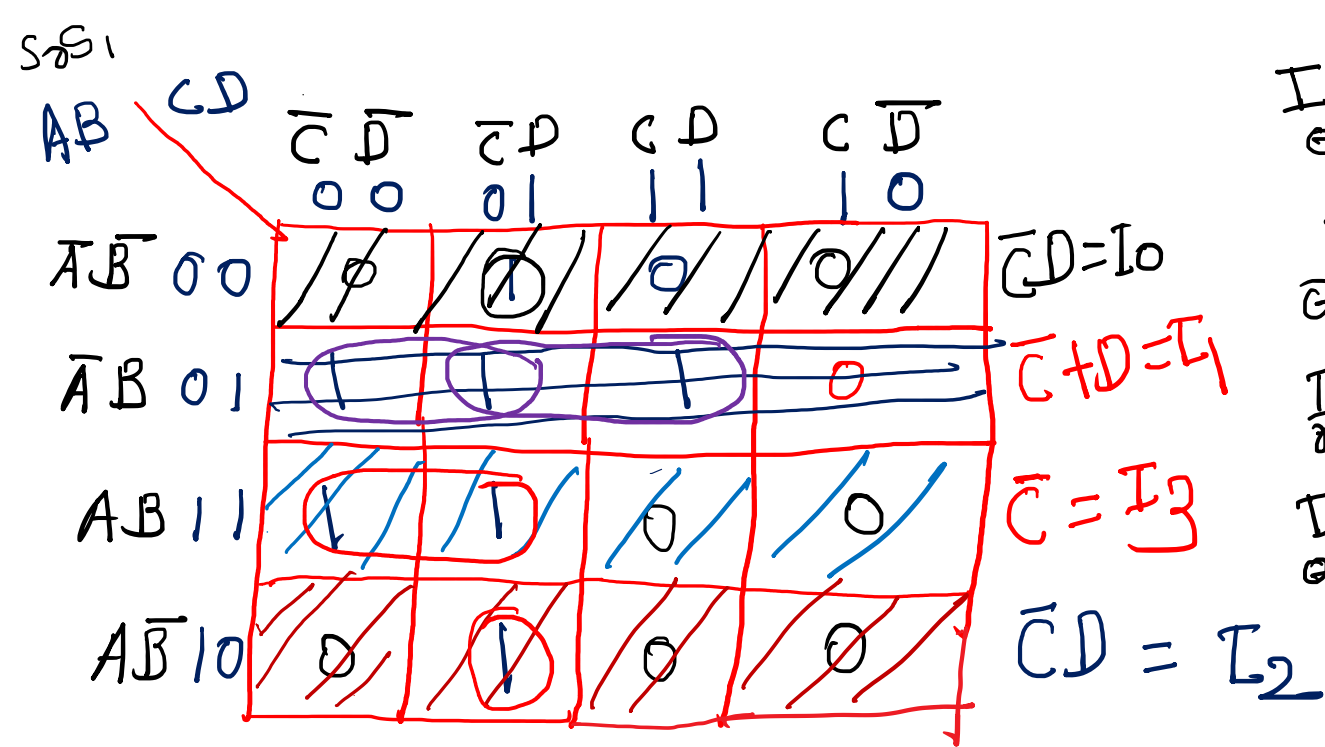
Step-2 $\rightarrow$ 4x1 MUX



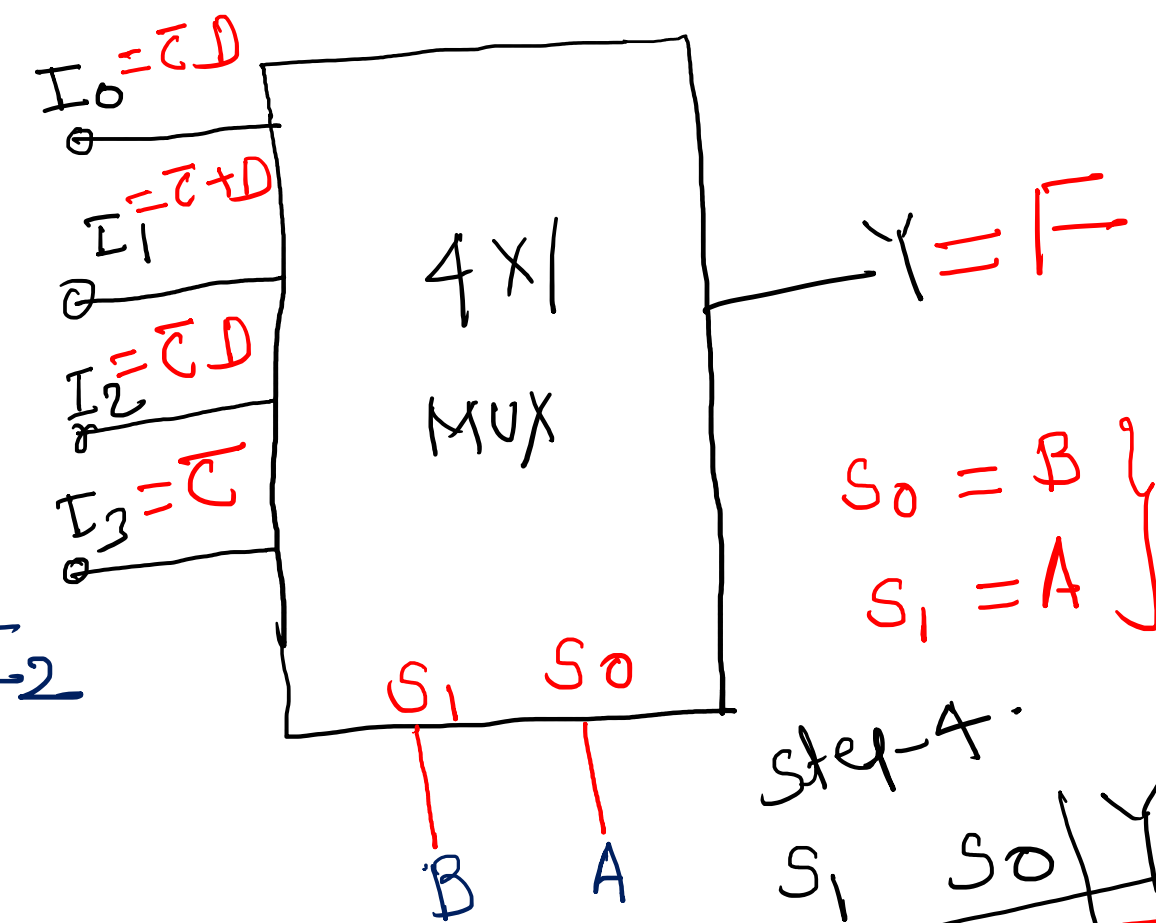
$S_0 = B$
 $S_1 = A$



Step-1 $\rightarrow$ K-Map.



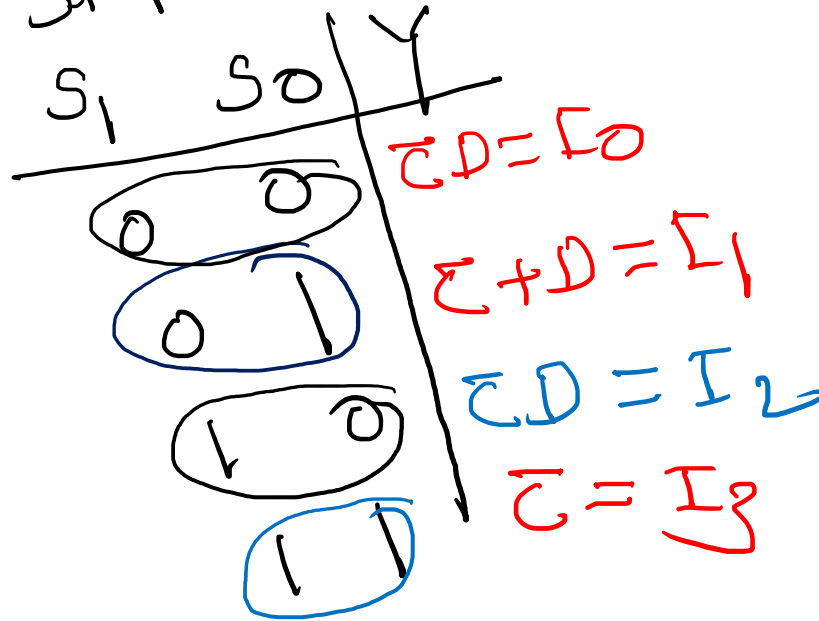
Step-2 $\rightarrow$ 4x1 MUX



$S_0 = B$
 $S_1 = A$

$S_L \rightarrow$ Step-3

Step-4.

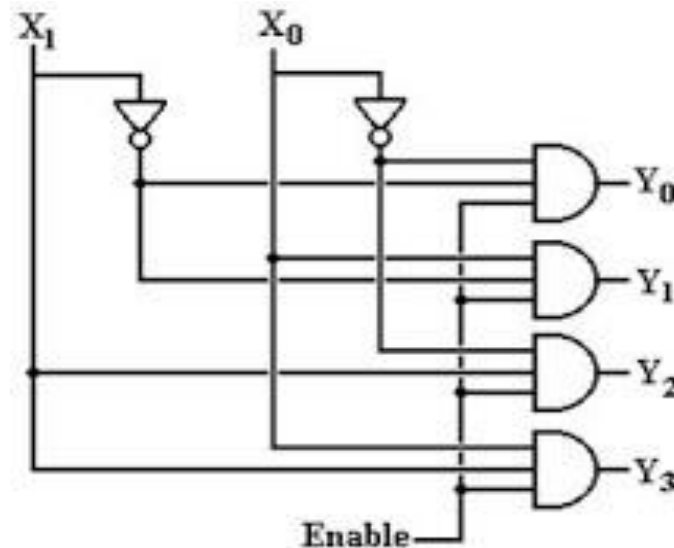


Exercise

1. Construct a 16x1 Multiplexer with two 8x1 and one 2x1 multiplexers. Write the Verilog HDL.
2. Implement the following Boolean function with a Multiplexer
 - a) $F(A,B,C,D) = \Sigma(0,2,5,8,10,14)$
 - b) $F(A,B,C,D) = \prod(2,6,11)$
3. Implement a full adder with two 4x1 Multiplexer

De-Multiplexer

- De-Mux is a combinational circuit that receives input from a single line and directs it to one of the 2^n output lines based on the 'n' bit selection lines.
- A decoder with enable input can function as a de-multiplexer with the following considerations.
- The input of decoder can be considered as selection lines of the de-multiplexer
- The enable signal of the decoder can be considered as the data input of the de-multiplexer

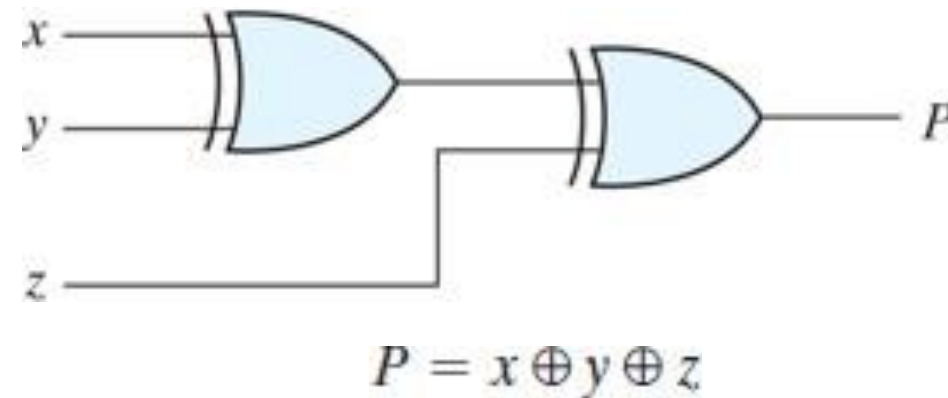


Even Parity Generator

- A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even.
- A circuit that generates the parity bit in the transmitter is called parity generator.
- It is very useful in systems requiring error detection and correction codes.

Even-Parity-Generator Truth Table

Three-Bit Message			Parity Bit
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

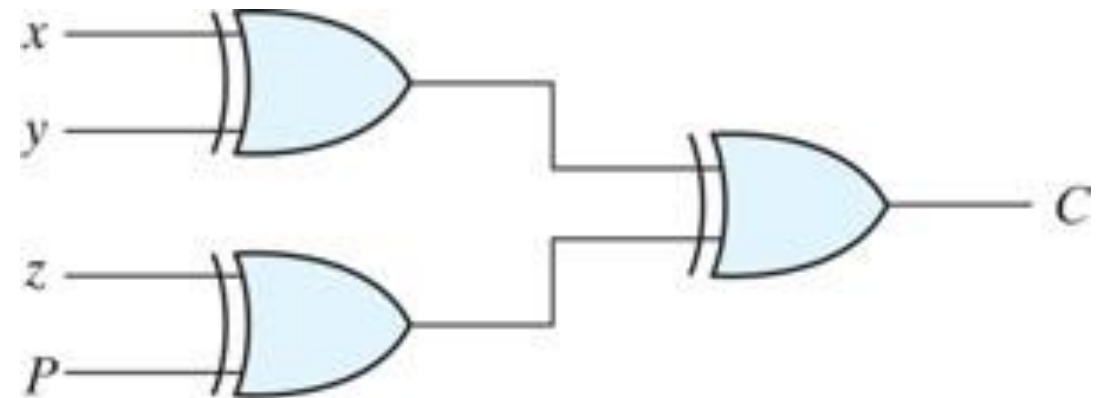


Even Parity Checker

- The circuit that checks the parity in the receiver is called a parity checker.

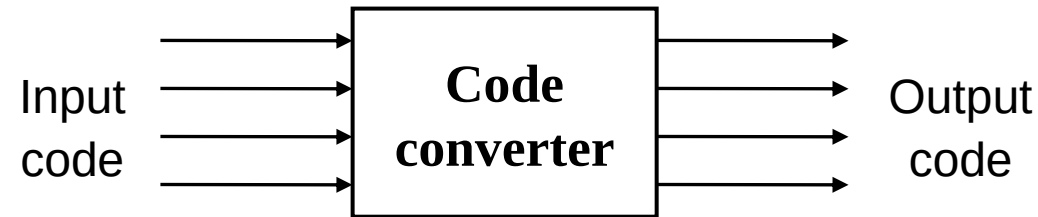
Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>	<i>C</i>
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



Code Converters

Code converters – take an input code, translate to its equivalent output code.



Example: **BCD to Excess-3 Code Converter.**

Input: BCD digit

Output: Excess-3 digit

BCD-to-Excess-3 Code converter

BCD is a code for the decimal digits 0-9

Excess-3 is also a code for the decimal digits

Decimal Digit	Input BCD	Output Excess-3
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Specification of BCD-to-Excess3

Inputs: a BCD input, A,B,C,D with A as the most significant bit and D as the least significant bit.

Outputs: an Excess-3 output W,X,Y,Z that corresponds to the BCD input.

Internal operation – circuit to do the conversion in combinational logic.

Formulation of BCD-to-Excess-3

Excess-3 code is easily formed by adding a binary 3 to the binary or BCD for the digit.

There are 16 possible inputs for both BCD and Excess-3.

It can be assumed that only valid BCD inputs will appear so the six combinations not used can be treated as don't cares.

BCD-to-Excess-3 Code Converter

■ Truth table:

	BCD				Excess-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
10	1	0	1	0	X	X	X	X
11	1	0	1	1	X	X	X	X
12	1	1	0	0	X	X	X	X
13	1	1	0	1	X	X	X	X
14	1	1	1	0	X	X	X	X
15	1	1	1	1	X	X	X	X

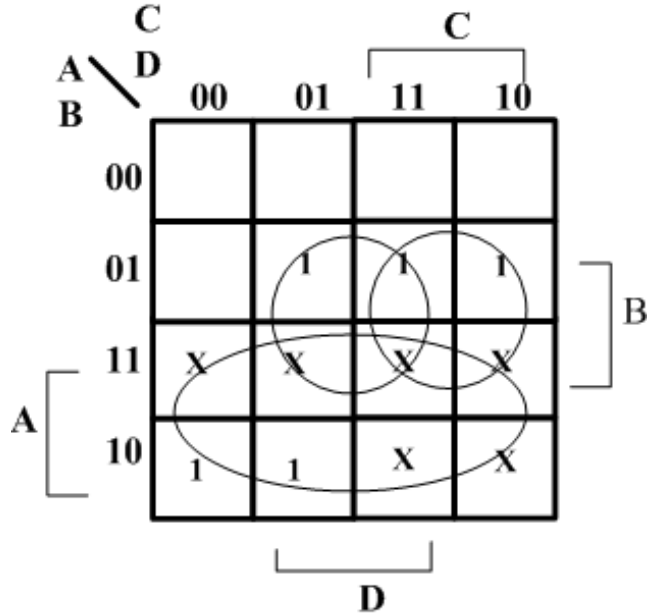
$$W(A,B,C,D) = \Sigma m(5,6,7,8,9) \\ + d(10,11,12,13,14,15)$$

$$X(A,B,C,D) = \Sigma m(1,2,3,4,9) \\ + d(10,11,12,13,14,15)$$

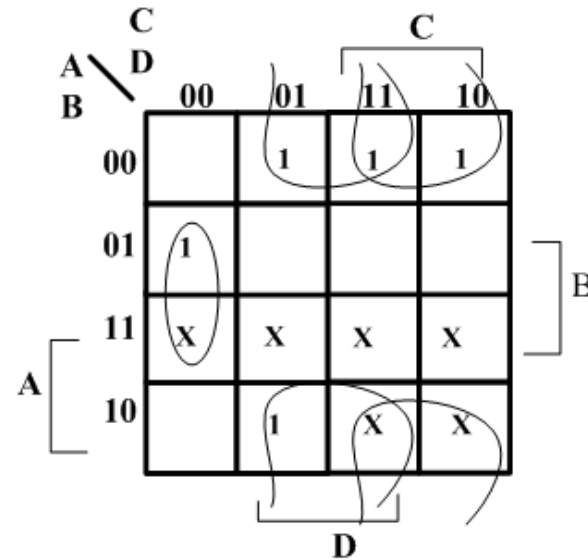
$$Y(A,B,C,D) = \Sigma m(0,3,4,7,8) \\ + d(10,11,12,13,14,15)$$

$$Z(A,B,C,D) = \Sigma m(0,2,4,6,8) \\ + d(10,11,12,13,14,15)$$

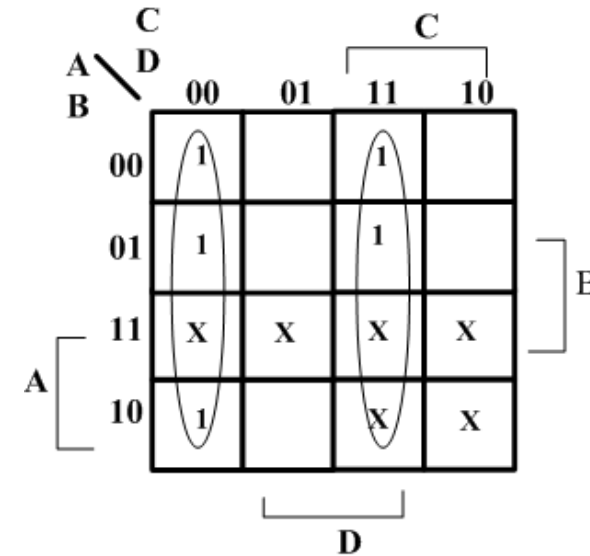
K-map for W



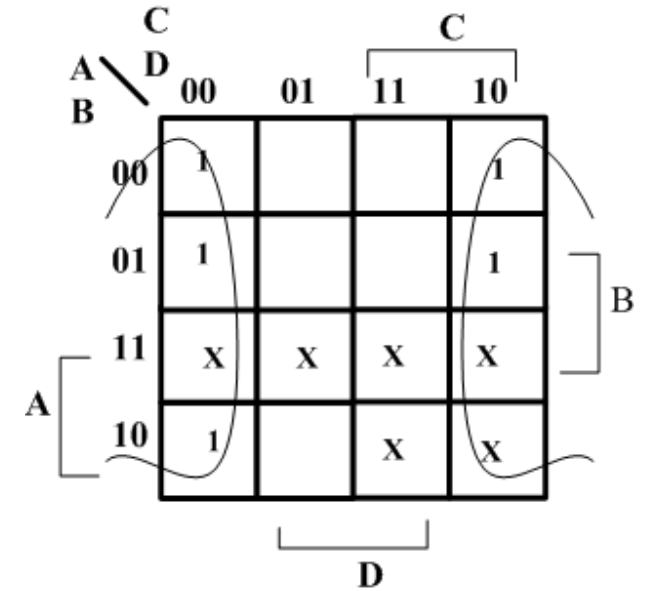
K-map for X



K-map for Y



K-map for Z



$$W(A,B,C,D) = \sum m(5,6,7,8,9) + d(10,11,12,13,14,15)$$

$$X(A,B,C,D) = \sum m(1,2,3,4,9) + d(10,11,12,13,14,15)$$

$$Y(A,B,C,D) = \sum m(0,3,4,7,8) + d(10,11,12,13,14,15)$$

$$Z(A,B,C,D) = \sum m(0,2,4,6,8) + d(10,11,12,13,14,15)$$

$$W = A + BC + BD$$

$$X = BC'D' + B'C + B'D$$

$$Y = CD + C'D'$$

$$Z = D'$$

Two level circuit implementation

Have equations

$$W = A + BC + BD = A + B(C+D)$$

$$X = B'C + B'D + BC'D' = B'(C+D) + BC'D'$$

$$Y = CD + C'D'$$

$$Z = D'$$

Factoring out (C+D) and call it T

Then $T' = (C+D)' = C'D'$

$$W = A + BT$$

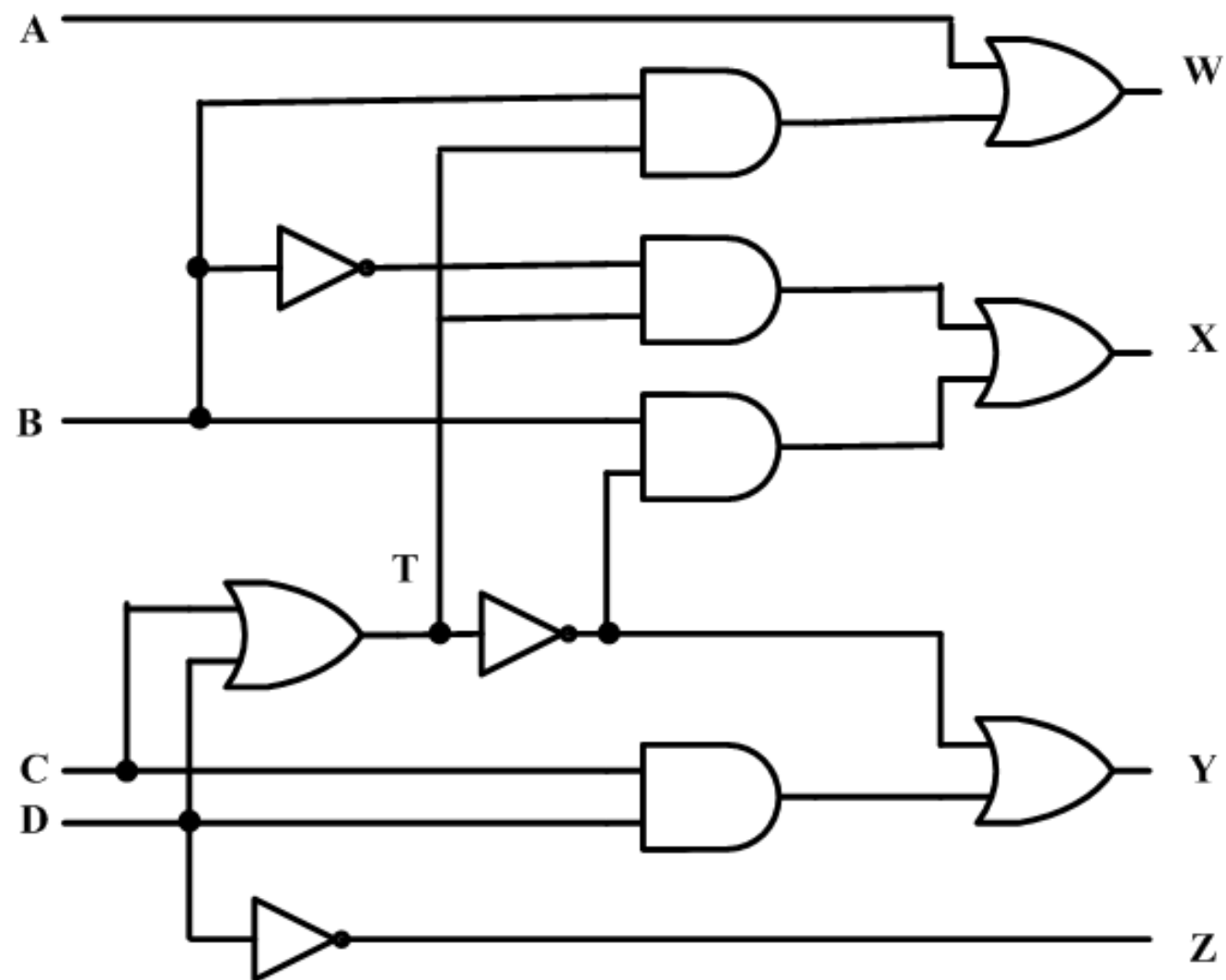
$$X = B'T + BT'$$

$$Y = CD + T'$$

$$Z = D'$$

Implementing the second set of equations where $T=C+D$ results in a lower gate count.

$$\begin{aligned}
 W &= A + BT \\
 X &= B'T + BT' \\
 Y &= CD + T' \\
 Z &= D'
 \end{aligned}$$



BCD-to-Seven-Segment Decoder

Specification

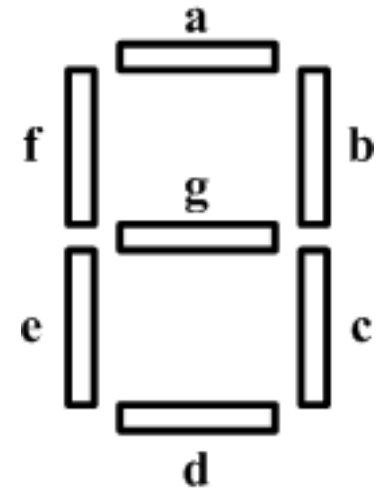
Each digit is created by lighting the appropriate segments. The segments are labeled a,b,c,d,e,f,g

The decoder takes a BCD input and outputs the correct code for the seven-segment display.

Input: A 4-bit binary value that is a BCD coded input.

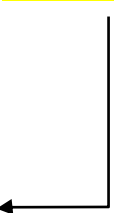
Outputs: 7 bits, a through g for each of the segments of the display.

Operation: Decode the input to activate the correct segments.



BCD Input					7 Segment Decoder						
Decimal	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1
10-15	All Other Inputs				0	0	0	0	0	0	0

Invalid BCD
codes
=
No Light



AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	0	0	0	0
10	1	1	0	0

a

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	0	0	0	0
10	1	1	0	0

b

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	0	0
10	1	1	0	0

c

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	0	0	0	0
10	1	1	0	0

d

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	0	0	0
10	1	0	0	0

e

AB \ CD	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	0	0	0	0
10	1	1	0	0

f

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	0	0	0	0
10	1	1	0	0

g

$$a = A'C + A'BD + AB'C' + B'C'D'$$

$$b = A'B' + A'C'D' + A'CD + B'C'$$

$$c = A'B + B'C' + A'C' + A'D$$

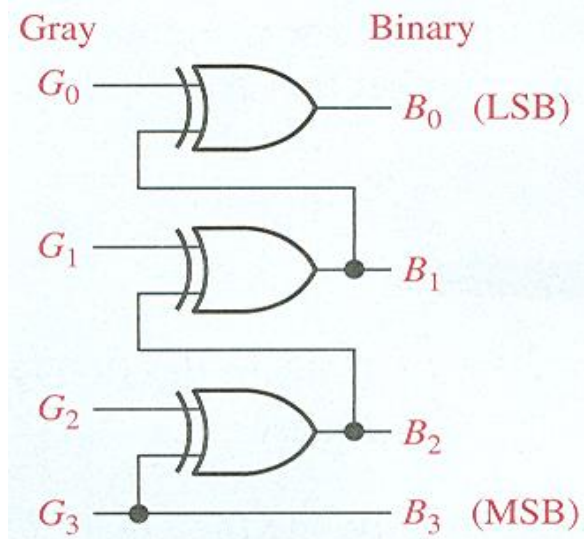
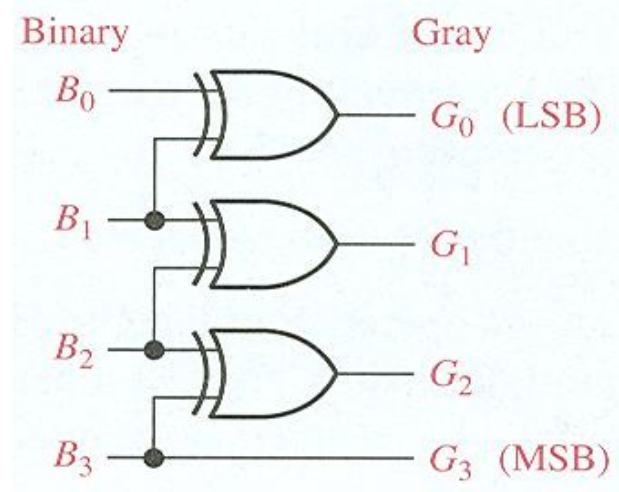
$$d = A'CD' + A'B'C + B'C'D' + AB'C' + A'BC'D$$

$$e = A'CD' + B'C'D'$$

$$f = A'BC' + A'C'D' + A'BD' + AB'C'$$

$$g = A'CD' + A'B'C + A'BC' + AB'C'$$

Binary-Gray conversions



BINARY	GRAY CODE
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101
1010	1111
1011	1110
1100	1010
1101	1011
1110	1001
1111	1000